Trabalho Prático #1
BaseBallX

Afonso Ferreira - 113480
Tomás Brás - 112665
David Pelicano - 113391

Web Semântica – 2024/2025
Mestrado em Engenharia Informática

Conteúdo

Lista de Figuras

Capítulo 1

Introdução ao Tema

1.1 Arquitetura da Aplicação e Tecnologias Utilizadas

O projeto BaseBallX tem como objetivo explorar dados históricos de baseball através de uma abordagem de Web Semântica. Em vez de trabalhar apenas com tabelas CSV isoladas, os dados foram convertidos para RDF e carregados num triple store, permitindo relacionar jogadores, equipas, franquias, salários, prémios, participações em All-Star Games, Hall of Fame e resultados de pós-temporada através de consultas SPARQL. Sobre esta base foi construída uma aplicação Web que suporta pesquisa, navegação por entidades, comparação de jogadores, visualização de relações e painéis estatísticos.

A solução encontra-se organizada em três camadas principais. A primeira corresponde à preparação dos dados, onde os ficheiros CSV são lidos e transformados para RDF. A segunda é a camada semântica, assegurada pelo GraphDB, onde o grafo resultante fica armazenado e disponível para interrogação. A terceira é a camada aplicacional, implementada em Django, responsável pela interface, autenticação, lógica de navegação e integração com o endpoint SPARQL.

- **Python** para os scripts de conversão de dados e ferramentas auxiliares do projeto.
- **rdflib** para criação das entidades RDF, definição de literais tipados e serialização do resultado em N3.
- **GraphDB** como triple store local e ponto de acesso SPARQL sobre o grafo produzido.
- **SPARQL** e **SPARQLWrapper** para consulta e integração dos dados semânticos na aplicação.
- **Django** como framework principal do backend Web, seguindo o padrão MVT para views, templates, autenticação e rotas.
- **SQLite** para persistência local de utilizadores, sessões e tentativas do quiz.
- **HTML**, **CSS**, **JavaScript**, **Bootstrap 5**, **Bootstrap Icons** e **Cytoscape.js** na camada de interface e visualização de grafos.

- **Bash** e **curl** em scripts auxiliares para automatizar o arranque do GraphDB e a criação/remoção de repositórios.

1.2 Fluxo Geral da Solução

O funcionamento global do sistema pode ser resumido nas seguintes etapas:

1. Os ficheiros CSV colocados na pasta `archive/` são usados como fonte primária de dados históricos de baseball.
2. O script `rdf/convert.py` lê esses ficheiros, normaliza identificadores, cria recursos RDF e escreve o resultado final em `rdf/baseball.n3`.
3. O ficheiro N3 gerado é carregado para um repositório `baseball` no GraphDB, onde passa a poder ser interrogado por SPARQL.
4. A aplicação Django consulta o endpoint local do GraphDB por SPARQL, usando a biblioteca `SPARQLWrapper` para obter os dados necessários para cada página.
5. A interface apresenta esses resultados ao utilizador em vistas de catálogo, detalhe, comparação, analytics e grafos de relações.

Esta organização permite separar claramente a transformação dos dados, o seu armazenamento semântico e a sua exploração na aplicação final, facilitando a manutenção e a extensão do projeto.

Capítulo 2

Dados, suas fontes e sua transformação

A base de conhecimento do projeto parte de um conjunto de ficheiros CSV com informação histórica sobre baseball, armazenados na pasta `archive/`. Estes dados cobrem entidades centrais, como jogadores, equipas e franquias, mas também estatísticas sazonais, salários, prémios e eventos especiais. A sua transformação para RDF permitiu passar de um conjunto de tabelas independentes para um grafo interligado, mais adequado para exploração semântica.

2.1 Fontes de Dados Principais

O núcleo do projeto assenta num arquivo histórico de baseball em formato CSV, compatível com a estrutura clássica da base de dados Lahman. No trabalho foram efetivamente utilizados os seguintes grupos de ficheiros:

- `Master.csv`: identificação dos jogadores e informação biográfica, como nome, data de nascimento, país de origem, altura, peso, datas de estreia e último jogo.
- `Teams.csv`, `TeamsFranchises.csv` e `TeamsHalf.csv`: informação sobre equipas, franquias, ligas e desempenho por época ou meia-época.
- `Batting.csv`, `BattingPost.csv`, `Pitching.csv`, `PitchingPost.csv`, `Fielding.csv` e `FieldingOF.csv`: estatísticas detalhadas de batting, pitching e fielding, tanto na fase regular como na pós-temporada.
- `Salaries.csv`: valores salariais por jogador, equipa e ano.
- `AwardsPlayers.csv` e `AwardsSharePlayers.csv`: prémios individuais e votações associadas aos jogadores.
- `AwardsManagers.csv` e `AwardsShareManagers.csv`: prémios e votações associados aos managers.
- `HallOfFame.csv` e `AllstarFull.csv`: votações para o Hall of Fame e participações em All-Star Games.

- `Managers.csv`, `ManagersHalf.csv` e `SeriesPost.csv`: histórico de managers e resultados de séries de pós-temporada.

Este conjunto de fontes foi suficiente para modelar um domínio rico, com relações entre pessoas, equipas, desempenhos desportivos, distinções individuais e contexto competitivo ao longo do tempo.

2.2 Transformação para RDF

O processo de transformação foi implementado no script `rdf/convert.py`. Esse script percorre cada CSV com `csv.DictReader`, extrai os atributos relevantes e cria recursos RDF com base no namespace `http://baseball.ws.pt/`. Para cada tipo de entidade é gerada uma URI estável e legível, sendo aplicadas regras de normalização para evitar caracteres problemáticos nos identificadores.

Durante a conversão são declaradas classes para jogadores, franquias, equipas, estatísticas desportivas, salários, prémios, Hall of Fame, All-Star Games e managers. Em seguida, cada linha dos CSV origina instâncias dessas classes e é ligada às restantes entidades através de propriedades semânticas. Por exemplo, um jogador pode ficar associado a batting, pitching, fielding, salários, prémios e aparições em All-Star Games, enquanto cada estatística fica ligada à equipa e ao ano respetivos.

- Os valores numéricos são convertidos para literais tipados, principalmente `xsd:integer` e `xsd:decimal`.
- Datas relevantes, como estreia e último jogo, são representadas com `xsd:date`.
- Campos lógicos, como vitória de divisão ou entrada no Hall of Fame, são convertidos para `xsd:boolean`.
- Valores vazios, inconsistentes ou marcados como `NA`, `null` e equivalentes são ignorados para evitar ruído no grafo.
- O nome legível do jogador é adicionalmente exposto com `foaf:name`, facilitando as consultas e a apresentação na interface.

O resultado final desta transformação é o ficheiro `rdf/baseball.n3`, em formato N3. Esse ficheiro é depois importado para um repositório `baseball` no GraphDB, onde passa a estar disponível para consulta através de SPARQL pela aplicação Django.

2.3 Dados Complementares para a Interface

Para além do grafo RDF principal, o projeto inclui uma pequena camada de enriquecimento visual para a interface. A partir do identificador `bbrefID` presente em `Master.csv`,

foram usados scripts auxiliares para construir catálogos JSON com fotografias de jogadores, recorrendo a URLs do Baseball-Reference e a imagens disponibilizadas pela MLB. Da mesma forma, os logótipos atuais das equipas são resolvidos através de identificadores estáticos da MLB.

Estes dados complementares não fazem parte do núcleo semântico armazenado no GraphDB, mas melhoram a experiência de utilização da aplicação, sobretudo nas páginas de detalhe, listagens de jogadores e visualizações gráficas.

Capítulo 3

Operações sobre os dados (SPARQL)

3.1 Visão geral da organização

As operações SPARQL do projeto foram reorganizadas por domínio funcional na pasta `webapp/baseball/sparql_queries/`. Esta divisão permite separar claramente as queries de jogadores, equipas, ligas, grafos, vistas auxiliares e quiz.

- `players.py`: catálogo de jogadores, filtros, detalhe biográfico, histórico desportivo, prémios, All-Star Games, Hall of Fame e salários.
- `teams.py`: diretório de franquias, histórico por época, postseason, plantéis, managers, All-Stars e prémios por equipa.
- `leagues.py`: navegação por liga, equipas da liga, séries e líderes estatísticos.
- `graphs.py`: queries usadas para construir o grafo relacional do detalhe de jogador.
- `misc.py`: listas auxiliares e rankings globais, como salários, Hall of Fame, managers e líderes agregados.
- `quiz.py`: banco de perguntas do quiz e verificação de respostas com `SELECT` e `ASK`.
- `edit_service.py`: operações `SPARQL UPDATE` para aprovar edições sobre jogadores e equipas.

O ficheiro `webapp/baseball/sparql.py` continua apenas como superfície de compatibilidade, reexportando as funções dos módulos internos.

3.2 Catálogo funcional das queries

Nesta secção, as operações são apresentadas no formato funcional pedido no enunciado: rota ou contexto de utilização, função Python associada, descrição e tipo de operação SPARQL.

3.2.1 Jogadores

- **/players/ – Contagem do catálogo**

  Função: `get_players_catalog_count()`

  Descrição: devolve o número total de jogadores que satisfazem os filtros ativos.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT (COUNT(DISTINCT ?player) AS ?total)
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name .
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
    BIND(
        CONCAT(
            LCASE(COALESCE(STR(?name), "")), " ",
            LCASE(COALESCE(STR(?nameFirst), "")), " ",
            LCASE(COALESCE(STR(?nameLast), "")), " ",
            LCASE(COALESCE(STR(?nameGiven), ""))
        ) AS ?searchBlob
    )
}
```

- **/players/ – Lista paginada de jogadores**

  Função: `get_players_catalog()`

  Descrição: devolve o subconjunto paginado do catálogo com os campos usados nos cartões da interface.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?bbrefID ?name ?nameFirst ?nameLast ?nameGiven
       ?birthCountry ?bats ?throws ?debut ?finalGame
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name .
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
    OPTIONAL { ?player bb:bbrefID ?bbrefID . }
    OPTIONAL { ?player bb:birthCountry ?birthCountry . }
    OPTIONAL { ?player bb:bats ?bats . }
    OPTIONAL { ?player bb:throws ?throws . }
    OPTIONAL { ?player bb:debut ?debut . }
    OPTIONAL { ?player bb:finalGame ?finalGame . }
}
ORDER BY ?name ?playerID
```

- **/players/ – Opções dos filtros**

  Função: `get_player_filter_options()`

  Descrição: devolve os valores usados pelos filtros de país de nascimento e década de estreia.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
SELECT DISTINCT ?birthCountry
WHERE {
    ?player a bb:Player ;
            bb:birthCountry ?birthCountry .
}
ORDER BY ?birthCountry
```

- **/players/?player1_letter=A e comparador – Lista de jogadores por inicial**

  Função: `get_player_options_by_initial(initial)`

  Descrição: devolve as opções de jogadores usadas nos *dropdowns* do comparador.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?name
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name .
    FILTER(STRSTARTS(UCASE(STR(?name)), "A"))
}
ORDER BY ?name ?playerID
```

- **/players/{player_id}/ – Resumo biográfico e agregado**

  Função: `get_player_summary()`

  Descrição: devolve os dados biográficos principais e alguns agregados globais do jogador.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?name ?nameFirst ?nameLast ?nameGiven ?playerID ?bbrefID
       ?birthCountry ?birthState ?birthCity ?bats ?throws ?debut ?finalGame
       ?awardsCount ?maxSalary
WHERE {
    ?player a bb:Player ;
            foaf:name ?name ;
            bb:playerID ?playerID .
    FILTER(?playerID = "aaronha01")
```

```
    OPTIONAL { ?player bb:bbrefID ?bbrefID . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
    OPTIONAL { ?player bb:birthCountry ?birthCountry . }
    OPTIONAL { ?player bb:birthState ?birthState . }
    OPTIONAL { ?player bb:birthCity ?birthCity . }
    OPTIONAL { ?player bb:bats ?bats . }
    OPTIONAL { ?player bb:throws ?throws . }
    OPTIONAL { ?player bb:debut ?debut . }
    OPTIONAL { ?player bb:finalGame ?finalGame . }
    OPTIONAL {
        SELECT (COUNT(DISTINCT ?awardObj) AS ?awardsCount)
        WHERE {
            ?player bb:playerID "aaronha01" ;
                    bb:wonAward ?awardObj .
        }
    }
    OPTIONAL {
        SELECT (MAX(?salaryValue) AS ?maxSalary)
        WHERE {
            ?player bb:playerID "aaronha01" ;
                    bb:hasSalary ?salaryObj .
            ?salaryObj bb:salary ?salaryValue .
        }
    }
}
LIMIT 1
```

- **/players/{player_id}/ – Histórico de batting**

  Função: `get_player_batting_seasons()`

  Descrição: devolve as linhas sazonais de batting do jogador.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?teamName ?lg
       ?games ?atBats ?hits ?homeRuns ?runsBattedIn
WHERE {
    ?player bb:playerID "aaronha01" ;
            bb:hasBatting ?batting .
    ?batting a bb:BattingStat .
    OPTIONAL { ?batting bb:yearID ?year . }
    OPTIONAL { ?batting bb:lgID ?lg . }
    OPTIONAL { ?batting bb:G ?games . }
    OPTIONAL { ?batting bb:AB ?atBats . }
    OPTIONAL { ?batting bb:H ?hits . }
    OPTIONAL { ?batting bb:HR ?homeRuns . }
    OPTIONAL { ?batting bb:RBI ?runsBattedIn . }
    OPTIONAL {
        ?batting bb:teamOf ?team .
        OPTIONAL { ?team bb:teamName ?teamName . }
    }
}
ORDER BY DESC(?year)
```

- **/players/{player_id}/ – Histórico de pitching**

  Função: `get_player_pitching_seasons()`

Descrição: devolve as linhas sazonais de pitching do jogador.

Tipo SPARQL: `SELECT`

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?teamName ?lg
       ?wins ?losses ?strikeouts ?inningsOuts ?era
WHERE {
    ?player bb:playerID "aaronha01" ;
            bb:hasPitching ?pitching .
    ?pitching a bb:PitchingStat .
    OPTIONAL { ?pitching bb:yearID ?year . }
    OPTIONAL { ?pitching bb:lgID ?lg . }
    OPTIONAL { ?pitching bb:W ?wins . }
    OPTIONAL { ?pitching bb:L ?losses . }
    OPTIONAL { ?pitching bb:SO ?strikeouts . }
    OPTIONAL { ?pitching bb:IPouts ?inningsOuts . }
    OPTIONAL { ?pitching bb:ERA ?era . }
    OPTIONAL {
        ?pitching bb:teamOf ?team .
        OPTIONAL { ?team bb:teamName ?teamName . }
    }
}
ORDER BY DESC(?year)
```

- **/players/{player_id}/ – Histórico de prémios**

  Função: `get_player_award_history()`

  Descrição: devolve os prémios ganhos pelo jogador ao longo da carreira.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?awardName ?lg ?notes
WHERE {
    ?player bb:playerID "aaronha01" ;
            bb:wonAward ?award .
    OPTIONAL { ?award bb:yearID ?year . }
    OPTIONAL { ?award bb:awardName ?awardName . }
    OPTIONAL { ?award bb:lgID ?lg . }
    OPTIONAL { ?award bb:notes ?notes . }
}
ORDER BY ?year ?awardName
```

- **/players/{player_id}/ – Histórico de All-Star**

  Função: `get_player_allstar_history()`

  Descrição: devolve as participações do jogador em All-Star Games.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?gameID ?startingPos ?teamName
WHERE {
    ?player bb:playerID "aaronha01" ;
            bb:playedInAllStar ?appearance .
    OPTIONAL { ?appearance bb:yearID ?year . }
    OPTIONAL { ?appearance bb:gameID ?gameID . }
    OPTIONAL { ?appearance bb:startingPos ?startingPos . }
    OPTIONAL {
        ?appearance bb:teamOf ?team .
        OPTIONAL { ?team bb:teamName ?teamName . }
    }
}
ORDER BY ?year
```

- **/players/{player_id}/ – Hall of Fame**

  Função: `get_player_hall_of_fame()`

  Descrição: devolve o histórico de votos do jogador para o Hall of Fame.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?inducted ?votes ?needed
WHERE {
    ?player bb:playerID "aaronha01" ;
            bb:hallOfFameVote ?vote .
    OPTIONAL { ?vote bb:yearID ?year . }
    OPTIONAL { ?vote bb:inducted ?inducted . }
    OPTIONAL { ?vote bb:votes ?votes . }
    OPTIONAL { ?vote bb:needed ?needed . }
}
ORDER BY ?year
```

- **/players/{player_id}/ – Histórico salarial**

  Função: `get_player_salary_history()`

  Descrição: devolve os salários do jogador por ano e equipa.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?salaryValue ?teamName
WHERE {
    ?player bb:playerID "aaronha01" ;
            bb:hasSalary ?salaryObj .
    ?salaryObj bb:salary ?salaryValue .
    OPTIONAL { ?salaryObj bb:yearID ?year . }
    OPTIONAL {
        ?salaryObj bb:teamOf ?team .
        OPTIONAL { ?team bb:teamName ?teamName . }
    }
}
ORDER BY DESC(?salaryValue) DESC(?year)
```

- **/players/{player_id}/ – Histórico de equipas**

  Função: `get_player_team_history()`

  Descrição: devolve as equipas e franquias pelas quais o jogador passou.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT DISTINCT ?year ?teamName ?franchiseName ?lg
WHERE {
    ?player bb:playerID "aaronha01" .
    {
        ?player bb:hasBatting ?season .
        ?season a bb:BattingStat .
    }
    UNION
    {
        ?player bb:hasPitching ?season .
        ?season a bb:PitchingStat .
    }
    ?season bb:yearID ?year .
    OPTIONAL { ?season bb:lgID ?lg . }
    OPTIONAL {
        ?season bb:teamOf ?team .
        OPTIONAL { ?team bb:teamName ?teamName . }
        OPTIONAL {
            ?team bb:franchiseOf ?franchise .
            OPTIONAL { ?franchise bb:franchiseName ?franchiseName . }
        }
    }
}
ORDER BY ?year ?teamName
```

3.2.2 Equipas e franquias

- **/teams/ – Diretório de franquias**

  Função: `get_team_franchise_catalog()`

  Descrição: constrói a grelha principal de equipas/franquias com informação da última época conhecida.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?franchiseId ?franchiseName ?active
       ?year ?teamName ?teamId ?teamIdBr ?league ?division ?rank
       ?park ?attendance ?wins ?losses
WHERE {
    ?franchise a bb:Franchise ;
               bb:franchID ?franchiseId .
    OPTIONAL { ?franchise bb:franchiseName ?franchiseName . }
    OPTIONAL { ?franchise bb:active ?active . }

    ?team a bb:Team ;
```

```
          bb:franchiseOf ?franchise ;
          bb:yearID ?year .

    OPTIONAL { ?team bb:teamName ?teamName . }
    OPTIONAL { ?team bb:teamID ?teamId . }
    OPTIONAL { ?team bb:teamIDBR ?teamIdBr . }
    OPTIONAL { ?team bb:lgID ?league . }
    OPTIONAL { ?team bb:divID ?division . }
    OPTIONAL { ?team bb:Rank ?rank . }
    OPTIONAL { ?team bb:park ?park . }
    OPTIONAL { ?team bb:attendance ?attendance . }
    OPTIONAL { ?team bb:W ?wins . }
    OPTIONAL { ?team bb:L ?losses . }
}
ORDER BY ?franchiseId DESC(?year)
```

- **/teams/{franchise_id}/ – Histórico sazonal da franquia**

  Função: `get_team_history()`

  Descrição: devolve as épocas da franquia com informação de contexto, recorde, batting, pitching e frequência.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?teamName ?teamId ?teamIdBr ?league ?division ?rank ?games ?homeGames
       ?wins ?losses ?runs ?attendance ?park
WHERE {
    ?franchise a bb:Franchise ;
               bb:franchID ?franchiseId .
    FILTER(?franchiseId = "ARI")

    ?team a bb:Team ;
          bb:franchiseOf ?franchise ;
          bb:yearID ?year .

    OPTIONAL { ?team bb:teamName ?teamName . }
    OPTIONAL { ?team bb:teamID ?teamId . }
    OPTIONAL { ?team bb:teamIDBR ?teamIdBr . }
    OPTIONAL { ?team bb:lgID ?league . }
    OPTIONAL { ?team bb:divID ?division . }
    OPTIONAL { ?team bb:Rank ?rank . }
    OPTIONAL { ?team bb:G ?games . }
    OPTIONAL { ?team bb:Ghome ?homeGames . }
    OPTIONAL { ?team bb:W ?wins . }
    OPTIONAL { ?team bb:L ?losses . }
    OPTIONAL { ?team bb:R ?runs . }
    OPTIONAL { ?team bb:attendance ?attendance . }
    OPTIONAL { ?team bb:park ?park . }
}
ORDER BY DESC(?year)
```

- **/teams/{franchise_id}/ – Histórico de pós-temporada**

  Função: `get_team_postseason_history()`

  Descrição: devolve as séries de postseason em que a franquia participou, com resultado, adversário e liga.

Tipo SPARQL: SELECT

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?round ?wins ?losses ?ties ?result ?teamName ?opponentName ?league ?opponentLeague
WHERE {
    ?franchise a bb:Franchise ;
               bb:franchID ?franchiseId .
    FILTER(?franchiseId = "ARI")

    ?series a bb:WorldSeriesResult ;
            bb:yearID ?year ;
            bb:round ?round ;
            bb:winnerTeam ?winnerTeam ;
            bb:loserTeam ?loserTeam .

    ?winnerTeam bb:franchiseOf ?winnerFranchise ;
                bb:teamName ?winnerTeamName .
    ?loserTeam bb:franchiseOf ?loserFranchise ;
               bb:teamName ?loserTeamName .

    FILTER(?winnerFranchise = ?franchise || ?loserFranchise = ?franchise)
}
ORDER BY DESC(?year) ?round
```

- **/teams/{franchise_id}/ – Plantel ofensivo da época**

  Função: `get_team_batting_roster()`

  Descrição: devolve os jogadores com linhas de batting associadas à equipa numa determinada época.

  Tipo SPARQL: SELECT

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?name
       (SUM(?games) AS ?games)
       (SUM(?atBats) AS ?atBats)
       (SUM(?hits) AS ?hits)
       (SUM(?homeRuns) AS ?homeRuns)
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name ;
            bb:hasBatting ?batting .
    ?batting bb:teamOf ?team .
    ?team bb:teamID ?teamId ;
          bb:yearID ?teamYear .
    FILTER(?teamId = "ARI" && ?teamYear = 2015)

    OPTIONAL { ?batting bb:G ?games . }
    OPTIONAL { ?batting bb:AB ?atBats . }
    OPTIONAL { ?batting bb:H ?hits . }
    OPTIONAL { ?batting bb:HR ?homeRuns . }
}
GROUP BY ?playerID ?name
ORDER BY DESC(?atBats) DESC(?homeRuns) ?name
```

- **/teams/{franchise_id}/ – Plantel de pitchers da época**

  Função: `get_team_pitching_roster()`

  Descrição: devolve os jogadores com linhas de pitching ligadas à equipa numa época concreta.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?name
       (SUM(?games) AS ?games)
       (SUM(?wins) AS ?wins)
       (SUM(?losses) AS ?losses)
       (SUM(?strikeouts) AS ?strikeouts)
       (SUM(?inningsOuts) AS ?inningsOuts)
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name ;
            bb:hasPitching ?pitching .
    ?pitching bb:teamOf ?team .
    ?team bb:teamID ?teamId ;
          bb:yearID ?teamYear .
    FILTER(?teamId = "ARI" && ?teamYear = 2015)

    OPTIONAL { ?pitching bb:G ?games . }
    OPTIONAL { ?pitching bb:W ?wins . }
    OPTIONAL { ?pitching bb:L ?losses . }
    OPTIONAL { ?pitching bb:SO ?strikeouts . }
    OPTIONAL { ?pitching bb:IPouts ?inningsOuts . }
}
GROUP BY ?playerID ?name
ORDER BY DESC(?inningsOuts) DESC(?strikeouts) ?name
```

- **/teams/{franchise_id}/ – Managers da época**

  Função: `get_team_managers()`

  Descrição: devolve os managers associados à equipa e os respetivos dados básicos de desempenho.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?playerID ?name ?wins ?losses ?rank ?inseason
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            bb:isManager ?manager .
    OPTIONAL { ?player foaf:name ?name . }

    ?manager bb:managedTeam ?team .
    ?team bb:teamID ?teamId ;
          bb:yearID ?teamYear .
```

```
    FILTER(?teamId = "ARI" && ?teamYear = 2015)

    OPTIONAL { ?manager bb:wins ?wins . }
    OPTIONAL { ?manager bb:losses ?losses . }
    OPTIONAL { ?manager bb:rank ?rank . }
    OPTIONAL { ?manager bb:inseason ?inseason . }
}
ORDER BY ?inseason ?rank ?name
```

- **/teams/{franchise_id}/ – All-Stars da equipa**

  Função: `get_team_all_stars()`

  Descrição: devolve os jogadores da equipa que participaram em All-Star Games nessa época.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?playerID ?name
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name ;
            bb:playedInAllStar ?appearance .
    ?appearance bb:teamOf ?team .
    ?team bb:teamID ?teamId ;
          bb:yearID ?teamYear .
    FILTER(?teamId = "ARI" && ?teamYear = 2015)
}
ORDER BY ?name
```

- **/teams/{franchise_id}/ – Prémios dos jogadores da equipa**

  Função: `get_team_awards()`

  Descrição: devolve os prémios ganhos por jogadores ligados à equipa nessa época.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT DISTINCT ?playerID ?name ?awardName
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name ;
            bb:wonAward ?award .
    ?award bb:yearID ?awardYear ;
           bb:awardName ?awardName .
    FILTER(?awardYear = 2015)

    {
        ?player bb:hasBatting ?line .
        ?line bb:teamOf ?team .
    }
```

```
    UNION
    {
        ?player bb:hasPitching ?line .
        ?line bb:teamOf ?team .
    }

    ?team bb:teamID ?teamId ;
          bb:yearID ?teamYear .
    FILTER(?teamId = "ARI" && ?teamYear = 2015)
}
ORDER BY ?awardName ?name
```

3.2.3 Ligas

- **Cabeçalho global da aplicação – Lista de ligas**

  Função: `get_header_leagues_graph()`

  Descrição: devolve as ligas ativas no grafo para construção da navegação superior.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT DISTINCT ?league
WHERE {
    ?team a bb:Team ;
          bb:lgID ?league .
    FILTER(STRLEN(STR(?league)) > 0)
}
ORDER BY ?league
```

- **/teams/league/{league_code}/ – Resumo da liga**

  Função: `get_league_detail()`

  Descrição: devolve contagens agregadas da liga, incluindo equipas, franquias e intervalo temporal.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT
    (COUNT(DISTINCT ?team) AS ?teamSeasons)
    (COUNT(DISTINCT ?franchise) AS ?franchises)
    (COUNT(DISTINCT ?year) AS ?seasons)
    (MIN(?year) AS ?firstYear)
    (MAX(?year) AS ?lastYear)
WHERE {
    ?team a bb:Team ;
          bb:lgID ?lg ;
          bb:yearID ?year .
    FILTER(?lg = "AL")
    OPTIONAL { ?team bb:franchID ?franchise . }
}
```

- **/teams/league/{league_code}/ – Equipas da liga**

  Função: `get_teams_by_league()`

  Descrição: devolve a lista de equipas/franquias da liga e a sua permanência temporal.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?teamName ?franchise (SAMPLE(?park) AS ?park)
       (COUNT(DISTINCT ?year) AS ?seasons)
       (MIN(?year) AS ?firstYear)
       (MAX(?year) AS ?lastYear)
WHERE {
    ?team a bb:Team ;
          bb:lgID ?lg ;
          bb:teamName ?teamName ;
          bb:yearID ?year .
    FILTER(?lg = "AL")
    OPTIONAL { ?team bb:franchID ?franchise . }
    OPTIONAL { ?team bb:park ?park . }
}
GROUP BY ?teamName ?franchise
ORDER BY ?teamName ?firstYear
```

- **/teams/league/{league_code}/ – Séries da liga**

  Função: `get_league_series_results()`

  Descrição: devolve as séries em que a liga participou como vencedora ou vencida.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?round ?winnerTeamName ?loserTeamName ?wins ?losses ?ties ?lgWinner ?lgLoser
WHERE {
    ?series a bb:WorldSeriesResult ;
            bb:yearID ?year ;
            bb:round ?round .
    ?series bb:winnerTeam ?winnerTeam .
    ?winnerTeam bb:teamName ?winnerTeamName .
    ?series bb:loserTeam ?loserTeam .
    ?loserTeam bb:teamName ?loserTeamName .
    OPTIONAL { ?series bb:wins ?wins . }
    OPTIONAL { ?series bb:losses ?losses . }
    OPTIONAL { ?series bb:ties ?ties . }
    OPTIONAL { ?series bb:lgIDwinner ?lgWinner . }
    OPTIONAL { ?series bb:lgIDloser ?lgLoser . }
    FILTER(STR(?lgWinner) = "AL" || STR(?lgLoser) = "AL")
}
ORDER BY DESC(?year) ?round
```

- **/teams/league/{league_code}/ – Líderes estatísticos**

  Função: `get_league_leaders()`

  Descrição: devolve os melhores jogadores da liga para uma estatística e ano concretos.

Tipo SPARQL: `SELECT`

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?statValue
WHERE {
    ?stat a bb:BattingStat ;
          bb:lgID "AL" ;
          bb:yearID 2015 ;
          bb:homeRuns ?statValue .
    ?player bb:hasBatting ?stat ;
            bb:playerID ?playerID .
    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
ORDER BY DESC(?statValue)
LIMIT 3
```

3.2.4 Grafo relacional

- **/players/{player_id}/ – Grafo do detalhe do jogador**

  Função: `get_player_graph_data(player_id)`

  Descrição: usa três queries separadas para obter o nó do jogador, as equipas/franquias ligadas ao jogador e os prémios conquistados.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT DISTINCT ?team ?teamName ?teamID ?year ?franchise ?franchiseName
WHERE {
    ?player bb:playerID "aaronha01" .
    {
        ?player bb:hasBatting ?line .
        ?line bb:teamOf ?team .
    }
    UNION
    {
        ?player bb:hasPitching ?line .
        ?line bb:teamOf ?team .
    }
    OPTIONAL { ?team bb:teamName ?teamName . }
    OPTIONAL { ?team bb:teamID ?teamID . }
    OPTIONAL { ?team bb:yearID ?year . }
    OPTIONAL {
        ?team bb:franchiseOf ?franchise .
        OPTIONAL { ?franchise bb:franchiseName ?franchiseName . }
    }
}
ORDER BY ?year ?teamName
```

3.2.5 Vistas auxiliares e rankings

- **/search/ – Pesquisa global de jogadores**

  Função: reutiliza `get_players_catalog()`

  Descrição: usa a query do catálogo de jogadores para compor os resultados da pesquisa global.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?name
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name .
}
ORDER BY ?name ?playerID
LIMIT 40
```

- **/search/ – Pesquisa global de equipas**

  Função: reutiliza `get_team_franchise_catalog()`

  Descrição: usa a query do diretório de franquias para compor os resultados de equipas na pesquisa.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?franchiseId ?franchiseName ?active
       ?year ?teamName ?teamId ?teamIdBr ?league ?division ?rank
WHERE {
    ?franchise a bb:Franchise ;
               bb:franchID ?franchiseId .
    OPTIONAL { ?franchise bb:franchiseName ?franchiseName . }
    OPTIONAL { ?franchise bb:active ?active . }
    ?team a bb:Team ;
          bb:franchiseOf ?franchise ;
          bb:yearID ?year .
}
ORDER BY ?franchiseId DESC(?year)
```

- **/salaries/ – Top salários**

  Função: `get_top_salaries()`

  Descrição: devolve os maiores salários registados.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?name ?salary ?year WHERE {
    ?p a bb:Player ;
       foaf:name ?name ;
       bb:hasSalary ?s .
    ?s bb:salary ?salary ;
       bb:yearID ?year .
}
ORDER BY DESC(?salary)
LIMIT 10
```

- **/awards/ – Lista de prémios**

  Função: `get_awards_list()`

  Descrição: devolve os prémios registados e o ano em que foram atribuídos.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
SELECT ?name ?award ?year
WHERE {
    ?p <http://xmlns.com/foaf/0.1/name> ?name .
    ?p <http://baseball.ws.pt/wonAward> ?awObj .
    ?awObj <http://baseball.ws.pt/awardName> ?award ;
           <http://baseball.ws.pt/yearID> ?year .
}
ORDER BY DESC(?year)
LIMIT 25
```

- **/halloffame/ – Hall of Fame**

  Função: `get_hall_of_fame_members()`

  Descrição: devolve os membros do Hall of Fame com dados de votos e ano de indução.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?yearID ?votedBy ?ballots ?votes ?needed
↪  ?category
WHERE {
    ?hof a bb:HallOfFameVote ;
         bb:inducted true ;
         bb:yearID ?yearID ;
         bb:category ?category ;
         bb:votes ?votes ;
         bb:ballots ?ballots .
    OPTIONAL { ?hof bb:votedBy ?votedBy . }
    OPTIONAL { ?hof bb:needed ?needed . }
    ?player bb:hallOfFameVote ?hof ;
            bb:playerID ?playerID .
    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
```

- **/managers/ – Lista de managers**

  Função: `get_managers_list()`

  Descrição: devolve os managers agregados por vitórias, derrotas e intervalo de anos.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name
       (SUM(?w) AS ?totalWins) (SUM(?l) AS ?totalLosses)
       (MIN(?yearID) AS ?firstYear) (MAX(?yearID) AS ?lastYear)
WHERE {
    ?mgr a bb:Manager ;
         bb:yearID ?yearID ;
         bb:wins ?w ;
         bb:losses ?l .
    ?player bb:isManager ?mgr ;
            bb:playerID ?playerID .
    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
GROUP BY ?playerID ?nameFirst ?nameLast ?nameGiven ?name
HAVING (SUM(?w) > 0)
ORDER BY DESC(?totalWins)
LIMIT 200
```

- **/analytics/ – Líderes globais de jogadores**

  Função: `get_global_player_leaders()`

  Descrição: devolve rankings globais agregados de jogadores para estatísticas acumuladas.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name (SUM(?statValue) AS ?totalStat)
WHERE {
    ?stat a bb:BattingStat ;
          bb:homeRuns ?statValue .
    ?player bb:hasBatting ?stat ;
            bb:playerID ?playerID .

    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
GROUP BY ?playerID ?nameFirst ?nameLast ?nameGiven ?name
ORDER BY DESC(?totalStat)
LIMIT 5
```

- **/analytics/ – Líderes globais de franquias**

  Função: `get_global_team_leaders()`

  Descrição: devolve rankings globais de franquias com base em vitórias acumuladas.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?franchID ?franchName (SUM(?w) AS ?totalWins)
WHERE {
    ?team a bb:Team ;
          bb:franchID ?franchID ;
          bb:W ?w .
    OPTIONAL { ?team bb:franchiseName ?franchName . }
}
GROUP BY ?franchID ?franchName
ORDER BY DESC(?totalWins)
LIMIT 10
```

- **Cabeçalho global da aplicação – Equipas**

  Função: `get_header_teams_graph()`

  Descrição: devolve equipas/franquias relevantes para o *header carousel*.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT DISTINCT ?teamCode ?teamName ?franchise
WHERE {
    ?team a bb:Team ;
          bb:yearID ?year ;
          bb:teamName ?teamName .
    FILTER(?year = 2015)
    OPTIONAL { ?team bb:teamIDBR ?teamCodeBR . }
    OPTIONAL { ?team bb:teamID ?teamCodeID . }
    OPTIONAL { ?team bb:franchID ?franchise . }
}
ORDER BY ?teamCode
```

3.2.6 Quiz interativo

- **/quiz/api/start/ – Banco de líderes estatísticos**

  Função: `get_quiz_leaderboard_bank()`

  Descrição: recolhe líderes reais por liga e ano para construir perguntas de ranking.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?lg ?year ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?statValue
WHERE {
    ?stat a bb:BattingStat ;
          bb:lgID ?lg ;
          bb:yearID ?year ;
          bb:homeRuns ?statValue .
    ?player bb:hasBatting ?stat ;
            bb:playerID ?playerID .
    FILTER(?year >= 2000)
    FILTER(?lg IN ("AL", "NL"))

    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
ORDER BY ?lg DESC(?year) DESC(?statValue) ?playerID
```

- **/quiz/api/start/ – Banco de salários**

  Função: `get_quiz_salary_bank()`

  Descrição: recolhe os maiores salários por ano para criar perguntas do tipo salary leader.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?year ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?salary
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            bb:hasSalary ?salaryObj .
    ?salaryObj bb:salary ?salary ;
               bb:yearID ?year .
    FILTER(?year >= 2000)
    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
ORDER BY DESC(?year) DESC(?salary) ?playerID
```

- **/quiz/api/start/ – Banco de vencedores de prémios**

  Função: `get_quiz_award_bank()`

  Descrição: recolhe vencedores reais de prémios para perguntas de múltipla escolha por ano.

  Tipo SPARQL: `SELECT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
```

```
SELECT ?awardName ?lg ?year ?playerID ?nameFirst ?nameLast ?nameGiven ?name
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            bb:wonAward ?award .
    ?award bb:awardName ?awardName ;
           bb:yearID ?year .
    FILTER(?year >= 2000)
    OPTIONAL { ?award bb:lgID ?lg . }
    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
ORDER BY ?awardName ?lg ?year ?playerID
```

- **/quiz/api/answer/ – Verificação de líderes estatísticos**

  Função: `ask_quiz_leaderboard_answer()`

  Descrição: confirma se a resposta escolhida é o verdadeiro líder dessa estatística.

  Tipo SPARQL: `ASK`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

ASK {
    ?player bb:playerID "arodial01" ;
            bb:hasBatting ?stat .
    ?stat a bb:BattingStat ;
          bb:lgID "AL" ;
          bb:yearID 2005 ;
          bb:homeRuns ?statValue .

    FILTER NOT EXISTS {
        ?otherStat a bb:BattingStat ;
                   bb:lgID "AL" ;
                   bb:yearID 2005 ;
                   bb:homeRuns ?otherValue .
        FILTER(?otherValue > ?statValue)
    }
}
```

- **/quiz/api/answer/ – Verificação de salário líder**

  Função: `ask_quiz_salary_answer()`

  Descrição: confirma se o jogador escolhido tem o salário mais alto registado nesse ano.

  Tipo SPARQL: `ASK`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

ASK {
    ?player a bb:Player ;
            bb:playerID "player01" ;
            bb:hasSalary ?salaryObj .
    ?salaryObj bb:salary ?salaryValue ;
               bb:yearID 2005 .
```

```
    FILTER NOT EXISTS {
        ?otherPlayer a bb:Player ;
                    bb:hasSalary ?otherSalaryObj .
        ?otherSalaryObj bb:salary ?otherValue ;
                        bb:yearID 2005 .
        FILTER(?otherValue > ?salaryValue)
    }
}
```

- **/quiz/api/answer/ – Verificação de vencedor de prémio**

  Função: `ask_quiz_award_answer()`

  Descrição: confirma se o jogador escolhido venceu realmente o prémio nesse ano.

  Tipo SPARQL: `ASK`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

ASK {
    ?player a bb:Player ;
            bb:playerID "santajo01" ;
            bb:wonAward ?award .
    ?award bb:awardName "Cy Young Award" ;
           bb:yearID 2004 ;
           bb:lgID "AL" .
}
```

3.2.7 Sugestões e edições administrativas

- **/suggestions/publish/ e aprovação de sugestões**

  Função: `_build_update_query()`

  Descrição: constroem instruções de atualização direta do GraphDB para editar valores de jogadores e equipas aprovados por administradores.

  Tipo SPARQL: `DELETE/INSERT`

  SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

DELETE { ?subject bb:birthCountry ?currentValue . }
INSERT { ?subject bb:birthCountry "USA" . }
WHERE {
  ?subject a bb:Player ; bb:playerID "aaronha01" .
  OPTIONAL { ?subject bb:birthCountry ?currentValue . }
}
```

3.3 Exemplos representativos

3.3.1 Exemplo 1: catálogo de jogadores

/players/ – Lista paginada de jogadores

Função: `get_players_catalog(...)`

Descrição: devolve um subconjunto paginado do catálogo, com filtros por nome, inicial, país de nascimento, *bats*, *throws*, década de estreia e disponibilidade de fotografia.

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?bbrefID ?name ?nameFirst ?nameLast ?nameGiven
       ?birthYear ?birthMonth ?birthDay ?birthCountry ?birthState ?birthCity
       ?deathYear ?deathMonth ?deathDay ?deathCountry ?deathState ?deathCity
       ?height ?weight ?bats ?throws ?debut ?finalGame
WHERE {
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name .
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
    OPTIONAL { ?player bb:bbrefID ?bbrefID . }
    OPTIONAL { ?player bb:birthYear ?birthYear . }
    OPTIONAL { ?player bb:birthMonth ?birthMonth . }
    OPTIONAL { ?player bb:birthDay ?birthDay . }
    OPTIONAL { ?player bb:birthCountry ?birthCountry . }
    OPTIONAL { ?player bb:birthState ?birthState . }
    OPTIONAL { ?player bb:birthCity ?birthCity . }
    OPTIONAL { ?player bb:deathYear ?deathYear . }
    OPTIONAL { ?player bb:deathMonth ?deathMonth . }
    OPTIONAL { ?player bb:deathDay ?deathDay . }
    OPTIONAL { ?player bb:deathCountry ?deathCountry . }
    OPTIONAL { ?player bb:deathState ?deathState . }
    OPTIONAL { ?player bb:deathCity ?deathCity . }
    OPTIONAL { ?player bb:height ?height . }
    OPTIONAL { ?player bb:weight ?weight . }
    OPTIONAL { ?player bb:bats ?bats . }
    OPTIONAL { ?player bb:throws ?throws . }
    OPTIONAL { ?player bb:debut ?debut . }
    OPTIONAL { ?player bb:finalGame ?finalGame . }
    BIND(
        CONCAT(
            LCASE(COALESCE(STR(?name), "")), " ",
            LCASE(COALESCE(STR(?nameFirst), "")), " ",
            LCASE(COALESCE(STR(?nameLast), "")), " ",
            LCASE(COALESCE(STR(?nameGiven), ""))
        ) AS ?searchBlob
    )
```

```
}
ORDER BY ?name ?playerID
LIMIT {limit}
OFFSET {offset}
```

3.3.2 Exemplo 2: histórico de franquia

/teams/{franchise_id}/ – Histórico completo da franquia

Função: `get_team_history(franchise_id)`

Descrição: devolve todas as épocas de uma franquia, com indicadores desportivos, batting, pitching, frequência e contexto competitivo.

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>

SELECT ?year ?teamName ?teamId ?teamIdBr ?league ?division ?rank ?games
↪  ?homeGames
      ?wins ?losses ?divisionWinner ?wildCardWinner ?leagueWinner
      ↪  ?worldSeriesWinner
      ?runs ?atBats ?hits ?doubles ?triples ?homeRuns ?walks ?strikeouts
      ↪  ?stolenBases
      ?caughtStealing ?hitByPitch ?sacrificeFlies ?runsAllowed ?earnedRuns ?era
      ?completeGames ?shutouts ?saves ?inningsOuts ?hitsAllowed
      ↪  ?homeRunsAllowed
      ?walksAllowed ?strikeoutsPitching ?errors ?doublePlays ?fieldPct ?park
      ?attendance ?bpf ?ppf
WHERE {
    ?franchise a bb:Franchise ;
               bb:franchID ?franchiseId .
    FILTER(?franchiseId = "{franchise_id}")

    ?team a bb:Team ;
          bb:franchiseOf ?franchise ;
          bb:yearID ?year .

    OPTIONAL { ?team bb:teamName ?teamName . }
    OPTIONAL { ?team bb:teamID ?teamId . }
    OPTIONAL { ?team bb:teamIDBR ?teamIdBr . }
    OPTIONAL { ?team bb:lgID ?league . }
    OPTIONAL { ?team bb:divID ?division . }
    OPTIONAL { ?team bb:Rank ?rank . }
    OPTIONAL { ?team bb:G ?games . }
    OPTIONAL { ?team bb:Ghome ?homeGames . }
    OPTIONAL { ?team bb:W ?wins . }
    OPTIONAL { ?team bb:L ?losses . }
```

```
    OPTIONAL { ?team bb:DivWin ?divisionWinner . }
    OPTIONAL { ?team bb:WCWin ?wildCardWinner . }
    OPTIONAL { ?team bb:LgWin ?leagueWinner . }
    OPTIONAL { ?team bb:WSWin ?worldSeriesWinner . }
    OPTIONAL { ?team bb:R ?runs . }
    OPTIONAL { ?team bb:AB ?atBats . }
    OPTIONAL { ?team bb:H ?hits . }
    OPTIONAL { ?team bb:2B ?doubles . }
    OPTIONAL { ?team bb:3B ?triples . }
    OPTIONAL { ?team bb:HR ?homeRuns . }
    OPTIONAL { ?team bb:BB ?walks . }
    OPTIONAL { ?team bb:SO ?strikeouts . }
    OPTIONAL { ?team bb:SB ?stolenBases . }
    OPTIONAL { ?team bb:CS ?caughtStealing . }
    OPTIONAL { ?team bb:HBP ?hitByPitch . }
    OPTIONAL { ?team bb:SF ?sacrificeFlies . }
    OPTIONAL { ?team bb:RA ?runsAllowed . }
    OPTIONAL { ?team bb:ER ?earnedRuns . }
    OPTIONAL { ?team bb:ERA ?era . }
    OPTIONAL { ?team bb:CG ?completeGames . }
    OPTIONAL { ?team bb:SHO ?shutouts . }
    OPTIONAL { ?team bb:SV ?saves . }
    OPTIONAL { ?team bb:IPouts ?inningsOuts . }
    OPTIONAL { ?team bb:HA ?hitsAllowed . }
    OPTIONAL { ?team bb:HRA ?homeRunsAllowed . }
    OPTIONAL { ?team bb:BBA ?walksAllowed . }
    OPTIONAL { ?team bb:SOA ?strikeoutsPitching . }
    OPTIONAL { ?team bb:E ?errors . }
    OPTIONAL { ?team bb:DP ?doublePlays . }
    OPTIONAL { ?team bb:FP ?fieldPct . }
    OPTIONAL { ?team bb:park ?park . }
    OPTIONAL { ?team bb:attendance ?attendance . }
    OPTIONAL { ?team bb:BPF ?bpf . }
    OPTIONAL { ?team bb:PPF ?ppf . }
}
ORDER BY DESC(?year)
```

3.3.3 Exemplo 3: verificação do quiz com ASK

/quiz/api/answer/ – Confirmação da resposta

Função: `ask_quiz_leaderboard_answer(stat_key, league_code, year, player_id)`

Descrição: confirma se o jogador escolhido é realmente o líder da estatística pedida nesse ano e nessa liga.

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
```

```
ASK {
    ?player bb:playerID "{player_id}" ;
            bb:{has_relation} ?stat .
    ?stat a bb:{stat_type_class} ;
          bb:lgID "{league_code}" ;
          bb:yearID {year} ;
          bb:{stat_property} ?statValue .

    FILTER NOT EXISTS {
        ?otherStat a bb:{stat_type_class} ;
                   bb:lgID "{league_code}" ;
                   bb:yearID {year} ;
                   bb:{stat_property} ?otherValue .
        FILTER(?otherValue > ?statValue)
    }
}
```

3.3.4 Exemplo 4: publicação de edições aprovadas

/suggestions/publish/ – Atualização direta do grafo

Função: `_build_update_query(...)`

Descrição: substitui o valor atual de um predicado pelo novo valor aprovado, mantendo a operação limitada ao sujeito correto.

SPARQL:

```
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

DELETE { ?subject {predicate} ?currentValue . }
INSERT { ?subject {predicate} {literal} . }
WHERE {
  {subject_selector}
  OPTIONAL { ?subject {predicate} ?currentValue . }
}
```

3.4 Inventário completo das queries

Para garantir que o relatório inclui **todas** as queries utilizadas pela aplicação, apresentam-se abaixo os módulos completos onde essas queries estão definidas. As funções auxiliares que apenas transformam resultados em Python não são o foco deste inventário; o objetivo é documentar integralmente as operações SPARQL efetivas do projeto.

3.4.1 players.py

```
from functools import lru_cache

from .base import (
    _build_batting_rates,
    _build_pitching_rates,
    _normalize_player_id,
    _row_float,
    _row_int,
    _row_value,
    escape_sparql_string,
    run_query,
)
@lru_cache(maxsize=64)
def get_player_options_by_initial(initial):
    initial = str(initial).strip().upper()
    if len(initial) != 1 or not initial.isalpha():
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?playerID ?name ?nameFirst ?nameLast
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                foaf:name ?name .
        OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
        OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
        FILTER(
            STRSTARTS(
                UCASE(STR(COALESCE(?nameFirst, ?name))),
                "{initial}"
            )
        )
    }}
    ORDER BY ?nameFirst ?nameLast ?name ?playerID
    """

    results = run_query(query)
    return [
        {
            "player_id": row["playerID"]["value"],
            "name": " ".join(
                part
                for part in [
                    _row_value(row, "nameFirst", ""),
                    _row_value(row, "nameLast", ""),
                ]
                if part
            ).strip()
            or row["name"]["value"],
        }
        for row in results
    ]


PLAYER_SORTS = {
    "name_asc": "ORDER BY ?name ?playerID",
    "name_desc": "ORDER BY DESC(?name) DESC(?playerID)",
    "debut_desc": "ORDER BY DESC(?debut) ?name ?playerID",
    "birth_year_desc": "ORDER BY DESC(?birthYear) ?name ?playerID",
}
```

```
def _player_catalog_filters(
    letter="",
    search_term="",
    birth_country="",
    bats="",
    throws="",
    debut_decade="",
    has_photo=False,
):
    filters = []

    if letter:
        safe_letter = escape_sparql_string(str(letter).strip().upper())
        if len(safe_letter) == 1 and safe_letter.isalpha():
            filters.append(
                'FILTER('
                f'STRSTARTS(UCASE(STR(COALESCE(?nameFirst, ?name))), "{safe_letter}")'
                ')'
            )

    if search_term:
        safe_search = escape_sparql_string(str(search_term).strip().lower())
        if safe_search:
            filters.append(f'FILTER(CONTAINS(LCASE(STR(?searchBlob)), "{safe_search}"))')

    if birth_country:
        safe_country = escape_sparql_string(str(birth_country).strip())
        if safe_country:
            filters.append(f'FILTER(?birthCountry = "{safe_country}")')

    if bats:
        safe_bats = escape_sparql_string(str(bats).strip().upper())
        if safe_bats in {"L", "R", "B"}:
            filters.append(f'FILTER(?bats = "{safe_bats}")')

    if throws:
        safe_throws = escape_sparql_string(str(throws).strip().upper())
        if safe_throws in {"L", "R"}:
            filters.append(f'FILTER(?throws = "{safe_throws}")')

    if debut_decade:
        try:
            decade = int(str(debut_decade).strip())
            filters.append(
                f'FILTER(?debut >= "{decade}-01-01"^^<http://www.w3.org/2001/XMLSchema#date> && '
                f'?debut < "{decade + 10}-01-01"^^<http://www.w3.org/2001/XMLSchema#date>)'
            )
        except ValueError:
            pass

    if has_photo:
        filters.append('FILTER(STRLEN(STR(?bbrefID)) > 0)')

    return "\n        ".join(filters)


@lru_cache(maxsize=256)
def get_players_catalog_count(
    letter="",
    search_term="",
    birth_country="",
    bats="",
    throws="",
```

```
    debut_decade="",
    has_photo=False,
):
    filters = _player_catalog_filters(
        letter,
        search_term,
        birth_country,
        bats,
        throws,
        debut_decade,
        has_photo,
    )
    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT (COUNT(DISTINCT ?player) AS ?total)
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                foaf:name ?name .
        OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
        OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
        OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
        OPTIONAL {{ ?player bb:bbrefID ?bbrefID . }}
        OPTIONAL {{ ?player bb:birthCountry ?birthCountry . }}
        OPTIONAL {{ ?player bb:bats ?bats . }}
        OPTIONAL {{ ?player bb:throws ?throws . }}
        OPTIONAL {{ ?player bb:debut ?debut . }}
        BIND(
            CONCAT(
                LCASE(COALESCE(STR(?name), "")), " ",
                LCASE(COALESCE(STR(?nameFirst), "")), " ",
                LCASE(COALESCE(STR(?nameLast), "")), " ",
                LCASE(COALESCE(STR(?nameGiven), ""))
            ) AS ?searchBlob
        )
        {filters}
    }}
    """

    results = run_query(query)
    if not results:
        return 0
    return _row_int(results[0], "total", 0)


@lru_cache(maxsize=256)
def get_players_catalog(
    letter="",
    search_term="",
    birth_country="",
    bats="",
    throws="",
    debut_decade="",
    has_photo=False,
    sort="name_asc",
    limit=24,
    offset=0,
):
    limit = max(int(limit), 1)
    offset = max(int(offset), 0)
    filters = _player_catalog_filters(
        letter,
        search_term,
```

```
    birth_country,
    bats,
    throws,
    debut_decade,
    has_photo,
)
sort_clause = PLAYER_SORTS.get(sort, PLAYER_SORTS["name_asc"])

query = f"""
PREFIX bb: <http://baseball.ws.pt/>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>

SELECT ?playerID ?bbrefID ?name ?nameFirst ?nameLast ?nameGiven
       ?birthYear ?birthMonth ?birthDay ?birthCountry ?birthState ?birthCity
       ?deathYear ?deathMonth ?deathDay ?deathCountry ?deathState ?deathCity
       ?height ?weight ?bats ?throws ?debut ?finalGame
WHERE {{
    ?player a bb:Player ;
            bb:playerID ?playerID ;
            foaf:name ?name .
    OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
    OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
    OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
    OPTIONAL {{ ?player bb:bbrefID ?bbrefID . }}
    OPTIONAL {{ ?player bb:birthYear ?birthYear . }}
    OPTIONAL {{ ?player bb:birthMonth ?birthMonth . }}
    OPTIONAL {{ ?player bb:birthDay ?birthDay . }}
    OPTIONAL {{ ?player bb:birthCountry ?birthCountry . }}
    OPTIONAL {{ ?player bb:birthState ?birthState . }}
    OPTIONAL {{ ?player bb:birthCity ?birthCity . }}
    OPTIONAL {{ ?player bb:deathYear ?deathYear . }}
    OPTIONAL {{ ?player bb:deathMonth ?deathMonth . }}
    OPTIONAL {{ ?player bb:deathDay ?deathDay . }}
    OPTIONAL {{ ?player bb:deathCountry ?deathCountry . }}
    OPTIONAL {{ ?player bb:deathState ?deathState . }}
    OPTIONAL {{ ?player bb:deathCity ?deathCity . }}
    OPTIONAL {{ ?player bb:height ?height . }}
    OPTIONAL {{ ?player bb:weight ?weight . }}
    OPTIONAL {{ ?player bb:bats ?bats . }}
    OPTIONAL {{ ?player bb:throws ?throws . }}
    OPTIONAL {{ ?player bb:debut ?debut . }}
    OPTIONAL {{ ?player bb:finalGame ?finalGame . }}
    BIND(
        CONCAT(
            LCASE(COALESCE(STR(?name), "")), " ",
            LCASE(COALESCE(STR(?nameFirst), "")), " ",
            LCASE(COALESCE(STR(?nameLast), "")), " ",
            LCASE(COALESCE(STR(?nameGiven), ""))
        ) AS ?searchBlob
    )
    {filters}
}}
{sort_clause}
LIMIT {limit}
OFFSET {offset}
"""

players = []
for row in run_query(query):
    first_name = _row_value(row, "nameFirst", "")
    last_name = _row_value(row, "nameLast", "")
    given_name = _row_value(row, "nameGiven", "")
    base_name = _row_value(row, "name", "Unknown Player")
    full_name = " ".join(part for part in [first_name, last_name] if part).strip()
    display_name = full_name or given_name or base_name
```

```
        players.append({
            "player_id": _row_value(row, "playerID", ""),
            "bbref_id": _row_value(row, "bbrefID", ""),
            "name": display_name,
            "name_first": first_name,
            "name_last": last_name,
            "name_given": given_name,
            "birth_year": _row_value(row, "birthYear", ""),
            "birth_month": _row_value(row, "birthMonth", ""),
            "birth_day": _row_value(row, "birthDay", ""),
            "birth_country": _row_value(row, "birthCountry", ""),
            "birth_state": _row_value(row, "birthState", ""),
            "birth_city": _row_value(row, "birthCity", ""),
            "death_year": _row_value(row, "deathYear", ""),
            "death_month": _row_value(row, "deathMonth", ""),
            "death_day": _row_value(row, "deathDay", ""),
            "death_country": _row_value(row, "deathCountry", ""),
            "death_state": _row_value(row, "deathState", ""),
            "death_city": _row_value(row, "deathCity", ""),
            "height": _row_value(row, "height", ""),
            "weight": _row_value(row, "weight", ""),
            "bats": _row_value(row, "bats", ""),
            "throws": _row_value(row, "throws", ""),
            "debut": _row_value(row, "debut", ""),
            "final_game": _row_value(row, "finalGame", ""),
        })
    return players


@lru_cache(maxsize=1)
def get_player_filter_options():
    country_query = """
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT DISTINCT ?birthCountry
    WHERE {
        ?player a bb:Player ;
                bb:birthCountry ?birthCountry .
        FILTER(STRLEN(STR(?birthCountry)) > 0)
    }
    ORDER BY ?birthCountry
    """

    decade_query = """
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT DISTINCT ?debut
    WHERE {
        ?player a bb:Player ;
                bb:debut ?debut .
    }
    ORDER BY ?debut
    """

    countries = [_row_value(row, "birthCountry", "") for row in run_query(country_query)]
    decades = []
    seen = set()
    for row in run_query(decade_query):
        debut = _row_value(row, "debut", "")
        try:
            year = int(str(debut)[:4])
        except (TypeError, ValueError):
            continue
        decade = year - (year % 10)
```

```
        if decade not in seen:
            seen.add(decade)
            decades.append(decade)

    return {
        "countries": [country for country in countries if country],
        "debut_decades": sorted(decades, reverse=True),
        "bats": [
            {"code": "R", "name": "Right"},
            {"code": "L", "name": "Left"},
            {"code": "B", "name": "Switch"},
        ],
        "throws": [
            {"code": "R", "name": "Right"},
            {"code": "L", "name": "Left"},
        ],
        "sorts": [
            {"code": "name_asc", "name": "Name A-Z"},
            {"code": "name_desc", "name": "Name Z-A"},
            {"code": "debut_desc", "name": "Latest debut"},
            {"code": "birth_year_desc", "name": "Youngest first"},
        ],
    }


def get_player_summary(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return None

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?name ?nameFirst ?nameLast ?nameGiven ?playerID ?bbrefID
           ?birthYear ?birthMonth ?birthDay ?birthCountry ?birthState ?birthCity
           ?deathYear ?deathMonth ?deathDay ?deathCountry ?deathState ?deathCity
           ?height ?weight ?bats ?throws ?debut ?finalGame
           ?awardsCount ?maxSalary ?careerHomeRuns ?careerRBI ?battingSeasons
    WHERE {{
        ?player a bb:Player ;
                foaf:name ?name ;
                bb:playerID ?playerID .
        FILTER(?playerID = "{player_id}")

        OPTIONAL {{ ?player bb:bbrefID ?bbrefID . }}
        OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
        OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
        OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
        OPTIONAL {{ ?player bb:birthYear ?birthYear . }}
        OPTIONAL {{ ?player bb:birthMonth ?birthMonth . }}
        OPTIONAL {{ ?player bb:birthDay ?birthDay . }}
        OPTIONAL {{ ?player bb:birthCountry ?birthCountry . }}
        OPTIONAL {{ ?player bb:birthState ?birthState . }}
        OPTIONAL {{ ?player bb:birthCity ?birthCity . }}
        OPTIONAL {{ ?player bb:deathYear ?deathYear . }}
        OPTIONAL {{ ?player bb:deathMonth ?deathMonth . }}
        OPTIONAL {{ ?player bb:deathDay ?deathDay . }}
        OPTIONAL {{ ?player bb:deathCountry ?deathCountry . }}
        OPTIONAL {{ ?player bb:deathState ?deathState . }}
        OPTIONAL {{ ?player bb:deathCity ?deathCity . }}
        OPTIONAL {{ ?player bb:height ?height . }}
        OPTIONAL {{ ?player bb:weight ?weight . }}
        OPTIONAL {{ ?player bb:bats ?bats . }}
        OPTIONAL {{ ?player bb:throws ?throws . }}
```

```
    OPTIONAL {{ ?player bb:debut ?debut . }}
    OPTIONAL {{ ?player bb:finalGame ?finalGame . }}

    OPTIONAL {{
        SELECT (COUNT(DISTINCT ?awardObj) AS ?awardsCount)
        WHERE {{
            ?player bb:playerID "{player_id}" ;
                    bb:wonAward ?awardObj .
        }}
    }}

    OPTIONAL {{
        SELECT (MAX(?salaryValue) AS ?maxSalary)
        WHERE {{
            ?player bb:playerID "{player_id}" ;
                    bb:hasSalary ?salaryObj .
            ?salaryObj bb:salary ?salaryValue .
        }}
    }}

    OPTIONAL {{
        SELECT
            (SUM(?homeRunsValue) AS ?careerHomeRuns)
            (SUM(?rbiValue) AS ?careerRBI)
            (COUNT(DISTINCT ?season) AS ?battingSeasons)
        WHERE {{
            ?player bb:playerID "{player_id}" ;
                    bb:hasBatting ?battingObj .
            ?battingObj a bb:BattingStat .
            OPTIONAL {{ ?battingObj bb:homeRuns ?homeRunsValue . }}
            OPTIONAL {{ ?battingObj bb:RBI ?rbiValue . }}
            OPTIONAL {{ ?battingObj bb:yearID ?season . }}
        }}
    }}
}}
LIMIT 1
"""

results = run_query(query)
if not results:
    return None

row = results[0]

def value_for(key, default="N/A"):
    return row.get(key, {}).get("value", default)

first_name = value_for("nameFirst", "")
last_name = value_for("nameLast", "")
given_name = value_for("nameGiven", "")
full_name = " ".join(part for part in [first_name, last_name] if part).strip()
if not full_name:
    full_name = value_for("name", "")
if not full_name:
    full_name = given_name or value_for("playerID")

return {
    "name": full_name,
    "name_first": first_name,
    "name_last": last_name,
    "name_given": given_name,
    "player_id": value_for("playerID"),
    "bbref_id": value_for("bbrefID", ""),
    "birth_year": value_for("birthYear"),
    "birth_month": value_for("birthMonth", ""),
```

```
        "birth_day": value_for("birthDay", ""),
        "birth_country": value_for("birthCountry"),
        "birth_state": value_for("birthState", ""),
        "birth_city": value_for("birthCity", ""),
        "death_year": value_for("deathYear", ""),
        "death_month": value_for("deathMonth", ""),
        "death_day": value_for("deathDay", ""),
        "death_country": value_for("deathCountry", ""),
        "death_state": value_for("deathState", ""),
        "death_city": value_for("deathCity", ""),
        "height": value_for("height"),
        "weight": value_for("weight"),
        "bats": value_for("bats"),
        "throws": value_for("throws"),
        "debut": value_for("debut"),
        "final_game": value_for("finalGame"),
        "awards_count": value_for("awardsCount", "0"),
        "max_salary": value_for("maxSalary", "0"),
        "career_home_runs": value_for("careerHomeRuns", "0"),
        "career_rbi": value_for("careerRBI", "0"),
        "batting_seasons": value_for("battingSeasons", "0"),
        "search_term": player_id,
    }


def get_player_batting_seasons(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?teamName ?lg
           ?gamesValue ?atBatsValue ?runsValue ?hitsValue ?doublesValue ?triplesValue
           ?homeRunsValue ?rbiValue ?stolenBasesValue ?walksValue ?strikeoutsValue
           ?hitByPitchValue ?sacrificeFliesValue
    WHERE {{
        ?player bb:playerID "{player_id}" ;
                bb:hasBatting ?batting .
        ?batting a bb:BattingStat ;
                 bb:yearID ?year .
        OPTIONAL {{ ?batting bb:G ?gamesValue . }}
        OPTIONAL {{ ?batting bb:AB ?atBatsValue . }}
        OPTIONAL {{ ?batting bb:R ?runsValue . }}
        OPTIONAL {{ ?batting bb:H ?hitsValue . }}
        OPTIONAL {{ ?batting <http://baseball.ws.pt/2B> ?doublesValue . }}
        OPTIONAL {{ ?batting <http://baseball.ws.pt/3B> ?triplesValue . }}
        OPTIONAL {{ ?batting bb:homeRuns ?homeRunsValue . }}
        OPTIONAL {{ ?batting bb:RBI ?rbiValue . }}
        OPTIONAL {{ ?batting bb:SB ?stolenBasesValue . }}
        OPTIONAL {{ ?batting bb:BB ?walksValue . }}
        OPTIONAL {{ ?batting bb:SO ?strikeoutsValue . }}
        OPTIONAL {{ ?batting bb:HBP ?hitByPitchValue . }}
        OPTIONAL {{ ?batting bb:SF ?sacrificeFliesValue . }}
        OPTIONAL {{
            ?batting bb:teamOf ?team .
            OPTIONAL {{ ?team bb:teamName ?teamName . }}
        }}
        OPTIONAL {{ ?batting bb:lgID ?lg . }}
    }}
    ORDER BY ?year
    """

    seasons = {}
```

```
    for row in run_query(query):
        year = _row_int(row, "year")
        if not year:
            continue

        season = seasons.setdefault(
            year,
            {
                "year": year,
                "teams": set(),
                "leagues": set(),
                "games": 0,
                "at_bats": 0,
                "runs": 0,
                "hits": 0,
                "doubles": 0,
                "triples": 0,
                "home_runs": 0,
                "rbi": 0,
                "stolen_bases": 0,
                "walks": 0,
                "strikeouts": 0,
                "hit_by_pitch": 0,
                "sacrifice_flies": 0,
            },
        )

        team_name = _row_value(row, "teamName")
        if team_name:
            season["teams"].add(team_name)
        lg = _row_value(row, "lg")
        if lg:
            season["leagues"].add(lg)

        season["games"] += _row_int(row, "gamesValue")
        season["at_bats"] += _row_int(row, "atBatsValue")
        season["runs"] += _row_int(row, "runsValue")
        season["hits"] += _row_int(row, "hitsValue")
        season["doubles"] += _row_int(row, "doublesValue")
        season["triples"] += _row_int(row, "triplesValue")
        season["home_runs"] += _row_int(row, "homeRunsValue")
        season["rbi"] += _row_int(row, "rbiValue")
        season["stolen_bases"] += _row_int(row, "stolenBasesValue")
        season["walks"] += _row_int(row, "walksValue")
        season["strikeouts"] += _row_int(row, "strikeoutsValue")
        season["hit_by_pitch"] += _row_int(row, "hitByPitchValue")
        season["sacrifice_flies"] += _row_int(row, "sacrificeFliesValue")

    lines = []
    for season in sorted(seasons.values(), key=lambda item: item["year"]):
        season["teams"] = sorted(season["teams"])
        season["leagues"] = sorted(season["leagues"])
        season["team_label"] = ", ".join(season["teams"]) if season["teams"] else "Unknown team"
        season["league_label"] = ", ".join(season["leagues"]) if season["leagues"] else "N/A"
        season["seasons"] = 1
        lines.append(_build_batting_rates(season))
    return lines


def get_player_batting_summary(player_id):
    seasons = get_player_batting_seasons(player_id)
    summary = {
        "games": 0,
        "at_bats": 0,
        "runs": 0,
```

```
        "hits": 0,
        "doubles": 0,
        "triples": 0,
        "home_runs": 0,
        "rbi": 0,
        "stolen_bases": 0,
        "walks": 0,
        "strikeouts": 0,
        "hit_by_pitch": 0,
        "sacrifice_flies": 0,
        "seasons": len(seasons),
    }

    for season in seasons:
        for key in (
            "games",
            "at_bats",
            "runs",
            "hits",
            "doubles",
            "triples",
            "home_runs",
            "rbi",
            "stolen_bases",
            "walks",
            "strikeouts",
            "hit_by_pitch",
            "sacrifice_flies",
        ):
            summary[key] += season.get(key, 0)

    return _build_batting_rates(summary)


def get_player_pitching_seasons(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?teamName ?lg
           ?winsValue ?lossesValue ?gamesValue ?gamesStartedValue ?savesValue
           ?inningsOutsValue ?earnedRunsValue ?strikeoutsValue ?eraValue
    WHERE {{
        ?player bb:playerID "{player_id}" ;
                bb:hasPitching ?pitching .
        ?pitching a bb:PitchingStat ;
                  bb:yearID ?year .
        OPTIONAL {{ ?pitching bb:W ?winsValue . }}
        OPTIONAL {{ ?pitching bb:L ?lossesValue . }}
        OPTIONAL {{ ?pitching bb:G ?gamesValue . }}
        OPTIONAL {{ ?pitching bb:GS ?gamesStartedValue . }}
        OPTIONAL {{ ?pitching bb:SV ?savesValue . }}
        OPTIONAL {{ ?pitching bb:IPouts ?inningsOutsValue . }}
        OPTIONAL {{ ?pitching bb:ER ?earnedRunsValue . }}
        OPTIONAL {{ ?pitching bb:SO ?strikeoutsValue . }}
        OPTIONAL {{ ?pitching bb:ERA ?eraValue . }}
        OPTIONAL {{
            ?pitching bb:teamOf ?team .
            OPTIONAL {{ ?team bb:teamName ?teamName . }}
        }}
        OPTIONAL {{ ?pitching bb:lgID ?lg . }}
    }}
```

```
ORDER BY ?year
"""

seasons = {}
for row in run_query(query):
    year = _row_int(row, "year")
    if not year:
        continue

    season = seasons.setdefault(
        year,
        {
            "year": year,
            "teams": set(),
            "leagues": set(),
            "wins": 0,
            "losses": 0,
            "games": 0,
            "games_started": 0,
            "saves": 0,
            "innings_outs": 0,
            "earned_runs": 0,
            "strikeouts": 0,
            "_has_wins": False,
            "_has_losses": False,
            "_has_games": False,
            "_has_games_started": False,
            "_has_saves": False,
            "_has_innings_outs": False,
            "_has_earned_runs": False,
            "_has_strikeouts": False,
            "_era_values": [],
        },
    )

    team_name = _row_value(row, "teamName")
    if team_name:
        season["teams"].add(team_name)
    lg = _row_value(row, "lg")
    if lg:
        season["leagues"].add(lg)

    if _row_value(row, "winsValue") not in (None, ""):
        season["wins"] += _row_int(row, "winsValue")
        season["_has_wins"] = True
    if _row_value(row, "lossesValue") not in (None, ""):
        season["losses"] += _row_int(row, "lossesValue")
        season["_has_losses"] = True
    if _row_value(row, "gamesValue") not in (None, ""):
        season["games"] += _row_int(row, "gamesValue")
        season["_has_games"] = True
    if _row_value(row, "gamesStartedValue") not in (None, ""):
        season["games_started"] += _row_int(row, "gamesStartedValue")
        season["_has_games_started"] = True
    if _row_value(row, "savesValue") not in (None, ""):
        season["saves"] += _row_int(row, "savesValue")
        season["_has_saves"] = True
    if _row_value(row, "inningsOutsValue") not in (None, ""):
        season["innings_outs"] += _row_int(row, "inningsOutsValue")
        season["_has_innings_outs"] = True
    if _row_value(row, "earnedRunsValue") not in (None, ""):
        season["earned_runs"] += _row_int(row, "earnedRunsValue")
        season["_has_earned_runs"] = True
    if _row_value(row, "strikeoutsValue") not in (None, ""):
        season["strikeouts"] += _row_int(row, "strikeoutsValue")
```

```
            season["_has_strikeouts"] = True
        if _row_value(row, "eraValue") not in (None, ""):
            season["_era_values"].append(_row_float(row, "eraValue", None))

    lines = []
    for season in sorted(seasons.values(), key=lambda item: item["year"]):
        season["teams"] = sorted(season["teams"])
        season["leagues"] = sorted(season["leagues"])
        season["team_label"] = ", ".join(season["teams"]) if season["teams"] else "Unknown team"
        season["league_label"] = ", ".join(season["leagues"]) if season["leagues"] else "N/A"
        season["seasons"] = 1
        if not season["_has_wins"]:
            season["wins"] = None
        if not season["_has_losses"]:
            season["losses"] = None
        if not season["_has_games"]:
            season["games"] = None
        if not season["_has_games_started"]:
            season["games_started"] = None
        if not season["_has_saves"]:
            season["saves"] = None
        if not season["_has_innings_outs"]:
            season["innings_outs"] = None
        if not season["_has_earned_runs"]:
            season["earned_runs"] = None
        if not season["_has_strikeouts"]:
            season["strikeouts"] = None
        if season["innings_outs"] not in (None, 0) and season["earned_runs"] is not None:
            season["era"] = (season["earned_runs"] * 27) / season["innings_outs"]
        else:
            era_values = [value for value in season["_era_values"] if value is not None]
            season["era"] = (sum(era_values) / len(era_values)) if era_values else None
        lines.append(_build_pitching_rates(season))
    return lines


def get_player_pitching_summary(player_id):
    seasons = get_player_pitching_seasons(player_id)
    summary = {"seasons": len(seasons)}

    for key in (
        "wins",
        "losses",
        "games",
        "games_started",
        "saves",
        "innings_outs",
        "earned_runs",
        "strikeouts",
    ):
        values = [season.get(key) for season in seasons if season.get(key) is not None]
        summary[key] = sum(values) if values else None

    if summary["innings_outs"] not in (None, 0) and summary["earned_runs"] is not None:
        summary["era"] = (summary["earned_runs"] * 27) / summary["innings_outs"]
    else:
        era_values = [season.get("era") for season in seasons if season.get("era") is not None]
        summary["era"] = (sum(era_values) / len(era_values)) if era_values else None
    return _build_pitching_rates(summary)


def get_player_award_history(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return []
```

```
    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?awardName ?lg ?notes
    WHERE {{
        ?player bb:playerID "{player_id}" ;
                bb:wonAward ?award .
        OPTIONAL {{ ?award bb:yearID ?year . }}
        OPTIONAL {{ ?award bb:awardName ?awardName . }}
        OPTIONAL {{ ?award bb:lgID ?lg . }}
        OPTIONAL {{ ?award bb:notes ?notes . }}
    }}
    ORDER BY ?year ?awardName
    """

    return [
        {
            "year": _row_int(row, "year", None),
            "award_name": _row_value(row, "awardName", "Award"),
            "league": _row_value(row, "lg", ""),
            "notes": _row_value(row, "notes", ""),
        }
        for row in run_query(query)
    ]


def get_player_allstar_history(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?gameID ?startingPos ?teamName
    WHERE {{
        ?player bb:playerID "{player_id}" ;
                bb:playedInAllStar ?appearance .
        OPTIONAL {{ ?appearance bb:yearID ?year . }}
        OPTIONAL {{ ?appearance bb:gameID ?gameID . }}
        OPTIONAL {{ ?appearance bb:startingPos ?startingPos . }}
        OPTIONAL {{
            ?appearance bb:teamOf ?team .
            OPTIONAL {{ ?team bb:teamName ?teamName . }}
        }}
    }}
    ORDER BY ?year
    """

    return [
        {
            "year": _row_int(row, "year", None),
            "game_id": _row_value(row, "gameID", ""),
            "starting_position": _row_int(row, "startingPos", None),
            "team_name": _row_value(row, "teamName", ""),
        }
        for row in run_query(query)
    ]


def get_player_hall_of_fame(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return {"inducted": False, "inducted_year": None, "vote_count": 0}
```

```
    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?inducted ?votes ?needed
    WHERE {{
        ?player bb:playerID "{player_id}" ;
                bb:hallOfFameVote ?vote .
        OPTIONAL {{ ?vote bb:yearID ?year . }}
        OPTIONAL {{ ?vote bb:inducted ?inducted . }}
        OPTIONAL {{ ?vote bb:votes ?votes . }}
        OPTIONAL {{ ?vote bb:needed ?needed . }}
    }}
    ORDER BY ?year
    """

    history = []
    inducted_year = None
    for row in run_query(query):
        inducted = str(_row_value(row, "inducted", "")).lower() == "true"
        year = _row_int(row, "year", None)
        if inducted and inducted_year is None:
            inducted_year = year
        history.append(
            {
                "year": year,
                "inducted": inducted,
                "votes": _row_int(row, "votes", None),
                "needed": _row_int(row, "needed", None),
            }
        )

    return {
        "inducted": inducted_year is not None,
        "inducted_year": inducted_year,
        "vote_count": len(history),
        "history": history,
    }


def get_player_salary_history(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?salaryValue ?teamName
    WHERE {{
        ?player bb:playerID "{player_id}" ;
                bb:hasSalary ?salaryObj .
        ?salaryObj bb:salary ?salaryValue .
        OPTIONAL {{ ?salaryObj bb:yearID ?year . }}
        OPTIONAL {{
            ?salaryObj bb:teamOf ?team .
            OPTIONAL {{ ?team bb:teamName ?teamName . }}
        }}
    }}
    ORDER BY DESC(?salaryValue) DESC(?year)
    """

    return [
        {
            "year": _row_int(row, "year", None),
```

```
            "salary": _row_int(row, "salaryValue"),
            "team_name": _row_value(row, "teamName", ""),
        }
        for row in run_query(query)
    ]


def get_player_team_history(player_id):
    player_id = _normalize_player_id(player_id)
    if not player_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT DISTINCT ?year ?teamName ?franchiseName ?lg
    WHERE {{
        ?player bb:playerID "{player_id}" .
        {{
            ?player bb:hasBatting ?season .
            ?season a bb:BattingStat .
        }}
        UNION
        {{
            ?player bb:hasPitching ?season .
            ?season a bb:PitchingStat .
        }}
        ?season bb:yearID ?year .
        OPTIONAL {{ ?season bb:lgID ?lg . }}
        OPTIONAL {{
            ?season bb:teamOf ?team .
            OPTIONAL {{ ?team bb:teamName ?teamName . }}
            OPTIONAL {{
                ?team bb:franchiseOf ?franchise .
                OPTIONAL {{ ?franchise bb:franchiseName ?franchiseName . }}
            }}
        }}
    }}
    ORDER BY ?year ?teamName
    """

    return [
        {
            "year": _row_int(row, "year", None),
            "team_name": _row_value(row, "teamName", ""),
            "franchise_name": _row_value(row, "franchiseName", ""),
            "league": _row_value(row, "lg", ""),
        }
        for row in run_query(query)
    ]
```

3.4.2 teams.py

```
from functools import lru_cache

from .base import (
    _build_batting_rates,
    _build_pitching_rates,
    _coerce_year_value,
    _row_bool,
    _row_float,
    _row_int,
    _row_value,
    escape_sparql_string,
```

```
    run_query,
)
def get_team_franchise_catalog():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?franchiseId ?franchiseName ?active
           ?year ?teamName ?teamId ?teamIdBr ?league ?division ?rank
           ?park ?attendance ?wins ?losses
           ?divisionWinner ?wildCardWinner ?leagueWinner ?worldSeriesWinner
    WHERE {
        ?franchise a bb:Franchise ;
                   bb:franchID ?franchiseId .
        OPTIONAL { ?franchise bb:franchiseName ?franchiseName . }
        OPTIONAL { ?franchise bb:active ?active . }

        ?team a bb:Team ;
              bb:franchiseOf ?franchise ;
              bb:yearID ?year .

        OPTIONAL { ?team bb:teamName ?teamName . }
        OPTIONAL { ?team bb:teamID ?teamId . }
        OPTIONAL { ?team bb:teamIDBR ?teamIdBr . }
        OPTIONAL { ?team bb:lgID ?league . }
        OPTIONAL { ?team bb:divID ?division . }
        OPTIONAL { ?team bb:Rank ?rank . }
        OPTIONAL { ?team bb:park ?park . }
        OPTIONAL { ?team bb:attendance ?attendance . }
        OPTIONAL { ?team bb:W ?wins . }
        OPTIONAL { ?team bb:L ?losses . }
        OPTIONAL { ?team bb:DivWin ?divisionWinner . }
        OPTIONAL { ?team bb:WCWin ?wildCardWinner . }
        OPTIONAL { ?team bb:LgWin ?leagueWinner . }
        OPTIONAL { ?team bb:WSWin ?worldSeriesWinner . }
    }
    ORDER BY ?franchiseId DESC(?year)
    """

    teams_map = {}
    for row in run_query(query):
        franchise_id = _row_value(row, "franchiseId", "")
        if not franchise_id:
            continue
        year = _row_int(row, "year", 0)
        entry = teams_map.setdefault(
            franchise_id,
            {
                "franchise_id": franchise_id,
                "name": _row_value(row, "franchiseName", "") or franchise_id,
                "active": _row_bool(row, "active", False),
                "season_count": 0,
                "first_year": year,
                "last_year": year,
                "latest_year": 0,
                "latest_team_name": "",
                "latest_team_id": "",
                "latest_team_id_br": "",
                "latest_league_code": "",
                "latest_division_code": "",
                "latest_rank": None,
                "latest_park": "",
                "latest_attendance": None,
                "latest_wins": None,
                "latest_losses": None,
                "latest_division_winner": False,
```

```
                "latest_wild_card_winner": False,
                "latest_league_winner": False,
                "latest_world_series_winner": False,
            },
        )
        entry["season_count"] += 1
        if year:
            entry["first_year"] = min(entry["first_year"], year) if entry["first_year"] else year
            entry["last_year"] = max(entry["last_year"], year)
        if year >= entry["latest_year"]:
            entry["latest_year"] = year
            entry["latest_team_name"] = _row_value(row, "teamName", "") or entry["name"]
            entry["latest_team_id"] = _row_value(row, "teamId", "")
            entry["latest_team_id_br"] = _row_value(row, "teamIdBr", "")
            entry["latest_league_code"] = _row_value(row, "league", "")
            entry["latest_division_code"] = _row_value(row, "division", "")
            entry["latest_rank"] = _row_int(row, "rank", None)
            entry["latest_park"] = _row_value(row, "park", "")
            entry["latest_attendance"] = _row_int(row, "attendance", None)
            entry["latest_wins"] = _row_int(row, "wins", None)
            entry["latest_losses"] = _row_int(row, "losses", None)
            entry["latest_division_winner"] = _row_bool(row, "divisionWinner", False)
            entry["latest_wild_card_winner"] = _row_bool(row, "wildCardWinner", False)
            entry["latest_league_winner"] = _row_bool(row, "leagueWinner", False)
            entry["latest_world_series_winner"] = _row_bool(row, "worldSeriesWinner", False)

    return sorted(
        teams_map.values(),
        key=lambda team: (
            0 if team["active"] else 1,
            team["name"],
            team["franchise_id"],
        ),
    )


def get_team_history(franchise_id):
    franchise_id = escape_sparql_string(str(franchise_id).strip().upper())
    if not franchise_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?teamName ?teamId ?teamIdBr ?league ?division ?rank ?games ?homeGames
           ?wins ?losses ?divisionWinner ?wildCardWinner ?leagueWinner ?worldSeriesWinner
           ?runs ?atBats ?hits ?doubles ?triples ?homeRuns ?walks ?strikeouts ?stolenBases
           ?caughtStealing ?hitByPitch ?sacrificeFlies ?runsAllowed ?earnedRuns ?era
           ?completeGames ?shutouts ?saves ?inningsOuts ?hitsAllowed ?homeRunsAllowed
           ?walksAllowed ?strikeoutsPitching ?errors ?doublePlays ?fieldPct ?park
           ?attendance ?bpf ?ppf
    WHERE {{
        ?franchise a bb:Franchise ;
                   bb:franchID ?franchiseId .
        FILTER(?franchiseId = "{franchise_id}")

        ?team a bb:Team ;
              bb:franchiseOf ?franchise ;
              bb:yearID ?year .

        OPTIONAL {{ ?team bb:teamName ?teamName . }}
        OPTIONAL {{ ?team bb:teamID ?teamId . }}
        OPTIONAL {{ ?team bb:teamIDBR ?teamIdBr . }}
        OPTIONAL {{ ?team bb:lgID ?league . }}
        OPTIONAL {{ ?team bb:divID ?division . }}
```

```
    OPTIONAL {{ ?team bb:Rank ?rank . }}
    OPTIONAL {{ ?team bb:G ?games . }}
    OPTIONAL {{ ?team bb:Ghome ?homeGames . }}
    OPTIONAL {{ ?team bb:W ?wins . }}
    OPTIONAL {{ ?team bb:L ?losses . }}
    OPTIONAL {{ ?team bb:DivWin ?divisionWinner . }}
    OPTIONAL {{ ?team bb:WCWin ?wildCardWinner . }}
    OPTIONAL {{ ?team bb:LgWin ?leagueWinner . }}
    OPTIONAL {{ ?team bb:WSWin ?worldSeriesWinner . }}
    OPTIONAL {{ ?team bb:R ?runs . }}
    OPTIONAL {{ ?team bb:AB ?atBats . }}
    OPTIONAL {{ ?team bb:H ?hits . }}
    OPTIONAL {{ ?team bb:2B ?doubles . }}
    OPTIONAL {{ ?team bb:3B ?triples . }}
    OPTIONAL {{ ?team bb:HR ?homeRuns . }}
    OPTIONAL {{ ?team bb:BB ?walks . }}
    OPTIONAL {{ ?team bb:SO ?strikeouts . }}
    OPTIONAL {{ ?team bb:SB ?stolenBases . }}
    OPTIONAL {{ ?team bb:CS ?caughtStealing . }}
    OPTIONAL {{ ?team bb:HBP ?hitByPitch . }}
    OPTIONAL {{ ?team bb:SF ?sacrificeFlies . }}
    OPTIONAL {{ ?team bb:RA ?runsAllowed . }}
    OPTIONAL {{ ?team bb:ER ?earnedRuns . }}
    OPTIONAL {{ ?team bb:ERA ?era . }}
    OPTIONAL {{ ?team bb:CG ?completeGames . }}
    OPTIONAL {{ ?team bb:SHO ?shutouts . }}
    OPTIONAL {{ ?team bb:SV ?saves . }}
    OPTIONAL {{ ?team bb:IPouts ?inningsOuts . }}
    OPTIONAL {{ ?team bb:HA ?hitsAllowed . }}
    OPTIONAL {{ ?team bb:HRA ?homeRunsAllowed . }}
    OPTIONAL {{ ?team bb:BBA ?walksAllowed . }}
    OPTIONAL {{ ?team bb:SOA ?strikeoutsPitching . }}
    OPTIONAL {{ ?team bb:E ?errors . }}
    OPTIONAL {{ ?team bb:DP ?doublePlays . }}
    OPTIONAL {{ ?team bb:FP ?fieldPct . }}
    OPTIONAL {{ ?team bb:park ?park . }}
    OPTIONAL {{ ?team bb:attendance ?attendance . }}
    OPTIONAL {{ ?team bb:BPF ?bpf . }}
    OPTIONAL {{ ?team bb:PPF ?ppf . }}
}}
ORDER BY DESC(?year)
"""

seasons = []
for row in run_query(query):
    season = {
        "year": _row_int(row, "year", 0),
        "team_name": _row_value(row, "teamName", "Unknown Team"),
        "team_id": _row_value(row, "teamId", ""),
        "team_id_br": _row_value(row, "teamIdBr", ""),
        "league_code": _row_value(row, "league", ""),
        "division_code": _row_value(row, "division", ""),
        "rank": _row_int(row, "rank", None),
        "games": _row_int(row, "games", 0),
        "home_games": _row_int(row, "homeGames", 0),
        "wins": _row_int(row, "wins", 0),
        "losses": _row_int(row, "losses", 0),
        "division_winner": _row_bool(row, "divisionWinner", False),
        "wild_card_winner": _row_bool(row, "wildCardWinner", False),
        "league_winner": _row_bool(row, "leagueWinner", False),
        "world_series_winner": _row_bool(row, "worldSeriesWinner", False),
        "runs": _row_int(row, "runs", 0),
        "at_bats": _row_int(row, "atBats", 0),
        "hits": _row_int(row, "hits", 0),
        "doubles": _row_int(row, "doubles", 0),
```

```
            "triples": _row_int(row, "triples", 0),
            "home_runs": _row_int(row, "homeRuns", 0),
            "walks": _row_int(row, "walks", 0),
            "strikeouts": _row_int(row, "strikeouts", 0),
            "stolen_bases": _row_int(row, "stolenBases", 0),
            "caught_stealing": _row_int(row, "caughtStealing", 0),
            "hit_by_pitch": _row_int(row, "hitByPitch", 0),
            "sacrifice_flies": _row_int(row, "sacrificeFlies", 0),
            "runs_allowed": _row_int(row, "runsAllowed", 0),
            "earned_runs": _row_int(row, "earnedRuns", 0),
            "era": _row_float(row, "era", None),
            "complete_games": _row_int(row, "completeGames", 0),
            "shutouts": _row_int(row, "shutouts", 0),
            "saves": _row_int(row, "saves", 0),
            "innings_outs": _row_int(row, "inningsOuts", 0),
            "hits_allowed": _row_int(row, "hitsAllowed", 0),
            "home_runs_allowed": _row_int(row, "homeRunsAllowed", 0),
            "walks_allowed": _row_int(row, "walksAllowed", 0),
            "strikeouts_pitching": _row_int(row, "strikeoutsPitching", 0),
            "errors": _row_int(row, "errors", 0),
            "double_plays": _row_int(row, "doublePlays", 0),
            "fielding_pct": _row_float(row, "fieldPct", None),
            "park": _row_value(row, "park", ""),
            "attendance": _row_int(row, "attendance", None),
            "bpf": _row_int(row, "bpf", None),
            "ppf": _row_int(row, "ppf", None),
            "seasons": 1,
        }
        season["win_pct"] = (season["wins"] / season["games"]) if season["games"] else None
        season["run_diff"] = season["runs"] - season["runs_allowed"]
        season["attendance_per_game"] = (
            season["attendance"] / season["home_games"]
            if season["attendance"] and season["home_games"]
            else None
        )
        _build_batting_rates(season)
        _build_pitching_rates(season)
        seasons.append(season)

    return seasons


def get_team_postseason_history(franchise_id):
    franchise_id = escape_sparql_string(str(franchise_id).strip().upper())
    if not franchise_id:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?round ?wins ?losses ?ties ?result ?teamName ?opponentName ?league ?opponentLeague
    WHERE {{
        ?franchise a bb:Franchise ;
                   bb:franchID ?franchiseId .
        FILTER(?franchiseId = "{franchise_id}")

        ?series a bb:WorldSeriesResult ;
                bb:yearID ?year ;
                bb:round ?round ;
                bb:winnerTeam ?winnerTeam ;
                bb:loserTeam ?loserTeam .

        ?winnerTeam bb:franchiseOf ?winnerFranchise ;
                    bb:teamName ?winnerTeamName .
        ?loserTeam bb:franchiseOf ?loserFranchise ;
```

```
                bb:teamName ?loserTeamName .

        OPTIONAL {{ ?winnerTeam bb:lgID ?winnerLeague . }}
        OPTIONAL {{ ?loserTeam bb:lgID ?loserLeague . }}
        OPTIONAL {{ ?series bb:wins ?wins . }}
        OPTIONAL {{ ?series bb:losses ?losses . }}
        OPTIONAL {{ ?series bb:ties ?ties . }}

        FILTER(?winnerFranchise = ?franchise || ?loserFranchise = ?franchise)

        BIND(IF(?winnerFranchise = ?franchise, "Won", "Lost") AS ?result)
        BIND(IF(?winnerFranchise = ?franchise, ?winnerTeamName, ?loserTeamName) AS ?teamName)
        BIND(IF(?winnerFranchise = ?franchise, ?loserTeamName, ?winnerTeamName) AS ?opponentName)
        BIND(IF(?winnerFranchise = ?franchise, ?winnerLeague, ?loserLeague) AS ?league)
        BIND(IF(?winnerFranchise = ?franchise, ?loserLeague, ?winnerLeague) AS ?opponentLeague)
    }}
    ORDER BY DESC(?year) ?round
    """

    results = []
    for row in run_query(query):
        results.append(
            {
                "year": _row_int(row, "year", None),
                "round": _row_value(row, "round", ""),
                "result": _row_value(row, "result", ""),
                "team_name": _row_value(row, "teamName", ""),
                "opponent_name": _row_value(row, "opponentName", ""),
                "wins": _row_int(row, "wins", 0),
                "losses": _row_int(row, "losses", 0),
                "ties": _row_int(row, "ties", 0),
                "league_code": _row_value(row, "league", ""),
                "opponent_league_code": _row_value(row, "opponentLeague", ""),
            }
        )
    return results


def get_team_batting_roster(team_code, year):
    team_code = escape_sparql_string(str(team_code).strip().upper())
    year_value = _coerce_year_value(year)
    if not team_code or year_value is None:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?playerID ?name
           (SUM(?games) AS ?games)
           (SUM(?atBats) AS ?atBats)
           (SUM(?hits) AS ?hits)
           (SUM(?doubles) AS ?doubles)
           (SUM(?triples) AS ?triples)
           (SUM(?homeRuns) AS ?homeRuns)
           (SUM(?runsBattedIn) AS ?runsBattedIn)
           (SUM(?stolenBases) AS ?stolenBases)
           (SUM(?walks) AS ?walks)
           (SUM(?strikeouts) AS ?strikeouts)
           (SUM(?hitByPitch) AS ?hitByPitch)
           (SUM(?sacrificeFlies) AS ?sacrificeFlies)
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                foaf:name ?name ;
```

```
                bb:hasBatting ?batting .
        ?batting bb:teamOf ?team .
        ?team bb:teamID ?teamId ;
              bb:yearID ?teamYear .
        FILTER(?teamId = "{team_code}" && ?teamYear = {year_value})

        OPTIONAL {{ ?batting bb:G ?games . }}
        OPTIONAL {{ ?batting bb:AB ?atBats . }}
        OPTIONAL {{ ?batting bb:H ?hits . }}
        OPTIONAL {{ ?batting bb:2B ?doubles . }}
        OPTIONAL {{ ?batting bb:3B ?triples . }}
        OPTIONAL {{ ?batting bb:HR ?homeRuns . }}
        OPTIONAL {{ ?batting bb:RBI ?runsBattedIn . }}
        OPTIONAL {{ ?batting bb:SB ?stolenBases . }}
        OPTIONAL {{ ?batting bb:BB ?walks . }}
        OPTIONAL {{ ?batting bb:SO ?strikeouts . }}
        OPTIONAL {{ ?batting bb:HBP ?hitByPitch . }}
        OPTIONAL {{ ?batting bb:SF ?sacrificeFlies . }}
    }}
    GROUP BY ?playerID ?name
    ORDER BY DESC(?atBats) DESC(?homeRuns) ?name
    """

    hitters = []
    for row in run_query(query):
        hitter = {
            "player_id": _row_value(row, "playerID", ""),
            "name": _row_value(row, "name", "Unknown Player"),
            "games": _row_int(row, "games", 0),
            "at_bats": _row_int(row, "atBats", 0),
            "hits": _row_int(row, "hits", 0),
            "doubles": _row_int(row, "doubles", 0),
            "triples": _row_int(row, "triples", 0),
            "home_runs": _row_int(row, "homeRuns", 0),
            "rbi": _row_int(row, "runsBattedIn", 0),
            "stolen_bases": _row_int(row, "stolenBases", 0),
            "walks": _row_int(row, "walks", 0),
            "strikeouts": _row_int(row, "strikeouts", 0),
            "hit_by_pitch": _row_int(row, "hitByPitch", 0),
            "sacrifice_flies": _row_int(row, "sacrificeFlies", 0),
            "seasons": 1,
        }
        hitter["plate_appearances"] = (
            hitter["at_bats"]
            + hitter["walks"]
            + hitter["hit_by_pitch"]
            + hitter["sacrifice_flies"]
        )
        _build_batting_rates(hitter)
        hitters.append(hitter)

    return hitters


def get_team_pitching_roster(team_code, year):
    team_code = escape_sparql_string(str(team_code).strip().upper())
    year_value = _coerce_year_value(year)
    if not team_code or year_value is None:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?playerID ?name
```

```
        (SUM(?games) AS ?games)
        (SUM(?wins) AS ?wins)
        (SUM(?losses) AS ?losses)
        (SUM(?gamesStarted) AS ?gamesStarted)
        (SUM(?saves) AS ?saves)
        (SUM(?strikeouts) AS ?strikeouts)
        (SUM(?inningsOuts) AS ?inningsOuts)
        (SUM(?earnedRuns) AS ?earnedRuns)
        (AVG(?eraValue) AS ?era)
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                foaf:name ?name ;
                bb:hasPitching ?pitching .
        ?pitching bb:teamOf ?team .
        ?team bb:teamID ?teamId ;
              bb:yearID ?teamYear .
        FILTER(?teamId = "{team_code}" && ?teamYear = {year_value})

        OPTIONAL {{ ?pitching bb:G ?games . }}
        OPTIONAL {{ ?pitching bb:W ?wins . }}
        OPTIONAL {{ ?pitching bb:L ?losses . }}
        OPTIONAL {{ ?pitching bb:GS ?gamesStarted . }}
        OPTIONAL {{ ?pitching bb:SV ?saves . }}
        OPTIONAL {{ ?pitching bb:SO ?strikeouts . }}
        OPTIONAL {{ ?pitching bb:IPouts ?inningsOuts . }}
        OPTIONAL {{ ?pitching bb:ER ?earnedRuns . }}
        OPTIONAL {{ ?pitching bb:ERA ?eraValue . }}
    }}
    GROUP BY ?playerID ?name
    ORDER BY DESC(?inningsOuts) DESC(?strikeouts) ?name
    """

    pitchers = []
    for row in run_query(query):
        pitcher = {
            "player_id": _row_value(row, "playerID", ""),
            "name": _row_value(row, "name", "Unknown Player"),
            "games": _row_int(row, "games", 0),
            "wins": _row_int(row, "wins", 0),
            "losses": _row_int(row, "losses", 0),
            "games_started": _row_int(row, "gamesStarted", 0),
            "saves": _row_int(row, "saves", 0),
            "strikeouts": _row_int(row, "strikeouts", 0),
            "innings_outs": _row_int(row, "inningsOuts", 0),
            "earned_runs": _row_int(row, "earnedRuns", 0),
            "era": _row_float(row, "era", None),
        }
        _build_pitching_rates(pitcher)
        pitchers.append(pitcher)

    return pitchers


def get_team_managers(team_code, year):
    team_code = escape_sparql_string(str(team_code).strip().upper())
    year_value = _coerce_year_value(year)
    if not team_code or year_value is None:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT DISTINCT ?playerID ?name ?wins ?losses ?rank ?inseason
```

```
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                bb:isManager ?manager .
        OPTIONAL {{ ?player foaf:name ?name . }}

        ?manager bb:managedTeam ?team .
        ?team bb:teamID ?teamId ;
              bb:yearID ?teamYear .
        FILTER(?teamId = "{team_code}" && ?teamYear = {year_value})

        OPTIONAL {{ ?manager bb:wins ?wins . }}
        OPTIONAL {{ ?manager bb:losses ?losses . }}
        OPTIONAL {{ ?manager bb:rank ?rank . }}
        OPTIONAL {{ ?manager bb:inseason ?inseason . }}
    }}
    ORDER BY ?inseason ?rank ?name
    """

    managers = []
    for row in run_query(query):
        managers.append(
            {
                "player_id": _row_value(row, "playerID", ""),
                "name": _row_value(row, "name", "Unknown Manager"),
                "wins": _row_int(row, "wins", 0),
                "losses": _row_int(row, "losses", 0),
                "rank": _row_int(row, "rank", None),
                "inseason": _row_value(row, "inseason", ""),
            }
        )
    return managers


def get_team_all_stars(team_code, year):
    team_code = escape_sparql_string(str(team_code).strip().upper())
    year_value = _coerce_year_value(year)
    if not team_code or year_value is None:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT DISTINCT ?playerID ?name
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                foaf:name ?name ;
                bb:playedInAllStar ?appearance .
        ?appearance bb:teamOf ?team .
        ?team bb:teamID ?teamId ;
              bb:yearID ?teamYear .
        FILTER(?teamId = "{team_code}" && ?teamYear = {year_value})
    }}
    ORDER BY ?name
    """

    return [
        {
            "player_id": _row_value(row, "playerID", ""),
            "name": _row_value(row, "name", "Unknown Player"),
        }
        for row in run_query(query)
    ]
```

```
def get_team_awards(team_code, year):
    team_code = escape_sparql_string(str(team_code).strip().upper())
    year_value = _coerce_year_value(year)
    if not team_code or year_value is None:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT DISTINCT ?playerID ?name ?awardName
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                foaf:name ?name ;
                bb:wonAward ?award .
        ?award bb:yearID ?awardYear ;
               bb:awardName ?awardName .
        FILTER(?awardYear = {year_value})

        {{
            ?player bb:hasBatting ?line .
            ?line bb:teamOf ?team .
        }}
        UNION
        {{
            ?player bb:hasPitching ?line .
            ?line bb:teamOf ?team .
        }}

        ?team bb:teamID ?teamId ;
              bb:yearID ?teamYear .
        FILTER(?teamId = "{team_code}" && ?teamYear = {year_value})
    }}
    ORDER BY ?awardName ?name
    """

    awards = []
    for row in run_query(query):
        awards.append(
            {
                "player_id": _row_value(row, "playerID", ""),
                "name": _row_value(row, "name", "Unknown Player"),
                "award_name": _row_value(row, "awardName", "Award"),
            }
        )
    return awards
```

3.4.3 leagues.py

```
from functools import lru_cache

from .base import _row_int, _row_value, escape_sparql_string, run_query
LEAGUE_LABELS = {
    "AL": "American League",
    "NL": "National League",
    "NA": "National Association",
    "AA": "American Association",
    "UA": "Union Association",
    "PL": "Players' League",
    "FL": "Federal League",
}
```

```
LEAGUE_ORDER = {
    "AL": 1,
    "NL": 2,
    "NA": 3,
    "AA": 4,
    "UA": 5,
    "PL": 6,
    "FL": 7,
}


DIVISION_LABELS = {
    "E": "East",
    "C": "Central",
    "W": "West",
}


def get_header_leagues_graph():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT DISTINCT ?league
    WHERE {
        ?team a bb:Team ;
              bb:lgID ?league .
        FILTER(STRLEN(STR(?league)) > 0)
    }
    ORDER BY ?league
    """

    leagues = []
    for row in run_query(query):
        code = _row_value(row, "league", "")
        if not code:
            continue
        leagues.append(
            {
                "code": code,
                "name": LEAGUE_LABELS.get(code, code),
                "slug": code.lower(),
                "sort_order": LEAGUE_ORDER.get(code, 999),
            }
        )

    return sorted(leagues, key=lambda league: (league["sort_order"], league["code"]))


def get_league_detail(league_code):
    league_code = escape_sparql_string(str(league_code).strip().upper())
    if not league_code:
        return None

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT
        (COUNT(DISTINCT ?team) AS ?teamSeasons)
        (COUNT(DISTINCT ?franchise) AS ?franchises)
        (COUNT(DISTINCT ?year) AS ?seasons)
        (MIN(?year) AS ?firstYear)
        (MAX(?year) AS ?lastYear)
    WHERE {{
```

```
        ?team a bb:Team ;
              bb:lgID ?lg ;
              bb:yearID ?year .
        FILTER(?lg = "{league_code}")
        OPTIONAL {{ ?team bb:franchID ?franchise . }}
    }}
    """

    results = run_query(query)
    if not results:
        return None

    row = results[0]
    team_seasons = _row_int(row, "teamSeasons", 0)
    if not team_seasons:
        return None

    return {
        "code": league_code,
        "name": LEAGUE_LABELS.get(league_code, league_code),
        "team_seasons": team_seasons,
        "franchises": _row_int(row, "franchises", 0),
        "seasons": _row_int(row, "seasons", 0),
        "first_year": _row_int(row, "firstYear", 0),
        "last_year": _row_int(row, "lastYear", 0),
    }


def get_teams_by_league(league_code):
    league_code = escape_sparql_string(str(league_code).strip().upper())
    if not league_code:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?teamName ?franchise (SAMPLE(?park) AS ?park)
           (COUNT(DISTINCT ?year) AS ?seasons)
           (MIN(?year) AS ?firstYear)
           (MAX(?year) AS ?lastYear)
    WHERE {{
        ?team a bb:Team ;
              bb:lgID ?lg ;
              bb:teamName ?teamName ;
              bb:yearID ?year .
        FILTER(?lg = "{league_code}")
        OPTIONAL {{ ?team bb:franchID ?franchise . }}
        OPTIONAL {{ ?team bb:park ?park . }}
    }}
    GROUP BY ?teamName ?franchise
    ORDER BY ?teamName ?firstYear
    """

    teams = []
    for row in run_query(query):
        teams.append(
            {
                "name": _row_value(row, "teamName", "Unknown Team"),
                "franchise": _row_value(row, "franchise", ""),
                "park": _row_value(row, "park", ""),
                "seasons": _row_int(row, "seasons", 0),
                "first_year": _row_int(row, "firstYear", 0),
                "last_year": _row_int(row, "lastYear", 0),
            }
        )
```

```
    return teams


def get_league_series_results(league_code):
    league_code = escape_sparql_string(str(league_code).strip().upper())
    if not league_code:
        return []

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?year ?round ?winnerTeamName ?loserTeamName ?wins ?losses ?ties ?lgWinner ?lgLoser
    WHERE {{
        ?series a bb:WorldSeriesResult ;
                bb:yearID ?year ;
                bb:round ?round .

        # --- FILTRO DE QUALIDADE ---
        # Ao remover o OPTIONAL daqui, as "Unknown Teams" desaparecem automaticamente
        ?series bb:winnerTeam ?winnerTeam .
        ?winnerTeam bb:teamName ?winnerTeamName .

        ?series bb:loserTeam ?loserTeam .
        ?loserTeam bb:teamName ?loserTeamName .
        # --------------------------

        OPTIONAL {{ ?series bb:wins ?wins . }}
        OPTIONAL {{ ?series bb:losses ?losses . }}
        OPTIONAL {{ ?series bb:ties ?ties . }}
        OPTIONAL {{ ?series bb:lgIDwinner ?lgWinner . }}
        OPTIONAL {{ ?series bb:lgIDloser ?lgLoser . }}

        FILTER(STR(?lgWinner) = "{league_code}" || STR(?lgLoser) = "{league_code}")
    }}
    ORDER BY DESC(?year) ?round
    """

    results = []
    for row in run_query(query):
        results.append({
            "year": _row_int(row, "year", None),
            "round": _row_value(row, "round", ""),
            "winner_team_name": _row_value(row, "winnerTeamName", "Unknown Team"),
            "loser_team_name": _row_value(row, "loserTeamName", "Unknown Team"),
            "wins": _row_int(row, "wins", 0),
            "losses": _row_int(row, "losses", 0),
            "ties": _row_int(row, "ties", 0),
            "winner_league": _row_value(row, "lgWinner", ""),
            "loser_league": _row_value(row, "lgLoser", ""),
        })
    return results


def get_league_leaders(league_code, year):
    league_code = escape_sparql_string(str(league_code).strip().upper())
    try:
        year_val = int(year)
    except (ValueError, TypeError):
        year_val = None
    if not league_code or not year_val:
        return {"hr": [], "rbi": [], "assists": [], "strikeouts": []}

    def _get_top_3(stat_type_class, stat_property, has_relation):
        query = f"""
        PREFIX bb: <http://baseball.ws.pt/>
```

```
        PREFIX foaf: <http://xmlns.com/foaf/0.1/>

        SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?statValue
        WHERE {{
            ?stat a bb:{stat_type_class} ;
                  bb:lgID "{league_code}" ;
                  bb:yearID {year_val} ;
                  bb:{stat_property} ?statValue .
            ?player bb:{has_relation} ?stat ;
                    bb:playerID ?playerID .

            OPTIONAL {{ ?player foaf:name ?name . }}
            OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
            OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
            OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
        }}
        ORDER BY DESC(?statValue)
        LIMIT 3
        """
        results = []
        for row in run_query(query):
            first_name = _row_value(row, "nameFirst", "")
            last_name = _row_value(row, "nameLast", "")
            full_name = " ".join(part for part in [first_name, last_name] if part).strip()
            if not full_name:
                full_name = _row_value(row, "name", "")
            if not full_name:
                full_name = _row_value(row, "nameGiven", _row_value(row, "playerID", ""))

            results.append({
                "name": full_name,
                "player_id": _row_value(row, "playerID", ""),
                "value": _row_int(row, "statValue", 0)
            })
        return results

    return {
        "hr": _get_top_3("BattingStat", "homeRuns", "hasBatting"),
        "rbi": _get_top_3("BattingStat", "RBI", "hasBatting"),
        "assists": _get_top_3("FieldingStat", "A", "hasFielding"),
        "strikeouts": _get_top_3("PitchingStat", "SO", "hasPitching"),
    }
```

3.4.4 graphs.py

```
from functools import lru_cache

from rdflib import Namespace
from rdflib.namespace import FOAF, RDF

from .base import escape_sparql_string, run_construct
from .leagues import LEAGUE_LABELS

BB = Namespace("http://baseball.ws.pt/")

NODE_TYPE_MAP = {
    BB.GraphPlayer: "player",
    BB.GraphFocusTeam: "focus-team",
    BB.GraphTeam: "team",
    BB.GraphFranchise: "franchise",
    BB.GraphAward: "award",
    BB.GraphLeague: "league",
    BB.GraphRosterPlayer: "player",
    BB.GraphTeammate: "teammate",
```

```
    BB.GraphManager: "manager",
}

NODE_TYPE_PRIORITY = {
    "player": 0,
    "focus-team": 1,
    "team": 2,
    "franchise": 3,
    "award": 4,
    "league": 5,
    "teammate": 6,
    "manager": 7,
}

EDGE_LABEL_MAP = {
    BB.playedFor: "played for",
    BB.franchiseLink: "franchise",
    BB.leagueLink: "league",
    BB.playedInLeague: "league",
    BB.rosterLink: "roster",
    BB.wonGraphAward: "won",
    BB.sharedClubhouseWith: "teammate",
    BB.managedByPerson: "managed by",
}


def _literal_value(graph, subject, *predicates):
    for predicate in predicates:
        value = graph.value(subject, predicate)
        if value is not None:
            return str(value)
    return ""


def _choose_node_type(graph, subject):
    node_types = [NODE_TYPE_MAP[obj] for obj in graph.objects(subject, RDF.type) if obj in NODE_TYPE_MAP]
    if not node_types:
        return None
    return sorted(node_types, key=lambda item: NODE_TYPE_PRIORITY.get(item, 999))[0]


def _build_node_label(graph, subject, node_type):
    if node_type in {"player", "teammate", "manager"}:
        return (
            _literal_value(graph, subject, FOAF.name, BB.nameGiven, BB.playerID)
            or "Unknown person"
        )

    if node_type in {"focus-team", "team"}:
        team_name = _literal_value(graph, subject, BB.teamName, FOAF.name, BB.teamID) or "Unknown team"
        season_year = _literal_value(graph, subject, BB.yearID)
        return f"{team_name} ({season_year})" if season_year else team_name

    if node_type == "franchise":
        return _literal_value(graph, subject, BB.franchiseName, BB.franchID) or "Unknown franchise"

    if node_type == "award":
        award_name = _literal_value(graph, subject, BB.awardName, FOAF.name) or "Award"
        award_year = _literal_value(graph, subject, BB.yearID)
        return f"{award_name} ({award_year})" if award_year else award_name

    if node_type == "league":
        league_code = _literal_value(graph, subject, BB.lgID)
        return LEAGUE_LABELS.get(league_code, league_code or "League")
```

```
    return _literal_value(graph, subject, FOAF.name) or str(subject)


def _player_graph_construct_query(player_id):
    return f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    CONSTRUCT {{
        ?player a bb:GraphPlayer ;
                bb:playerID ?playerCode ;
                foaf:name ?playerName .

        ?team a bb:GraphTeam ;
              bb:teamName ?teamName ;
              bb:teamID ?teamCode ;
              bb:yearID ?teamYear .

        ?franchise a bb:GraphFranchise ;
                   bb:franchiseName ?franchiseName ;
                   bb:franchID ?franchiseCode .

        ?award a bb:GraphAward ;
               bb:awardName ?awardName ;
               bb:yearID ?awardYear .

        ?leagueNode a bb:GraphLeague ;
                    bb:lgID ?leagueCode .

        ?teammate a bb:GraphTeammate ;
                  bb:playerID ?teammateCode ;
                  foaf:name ?teammateName ;
                  bb:sharedSeasons ?sharedSeasons .

        ?managerPlayer a bb:GraphManager ;
                       bb:playerID ?managerCode ;
                       foaf:name ?managerName .

        ?player bb:playedFor ?team .
        ?team bb:franchiseLink ?franchise .
        ?team bb:leagueLink ?leagueNode .
        ?player bb:playedInLeague ?leagueNode .
        ?player bb:wonGraphAward ?award .
        ?player bb:sharedClubhouseWith ?teammate .
        ?team bb:managedByPerson ?managerPlayer .
    }}
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerCode .
        FILTER(?playerCode = "{player_id}")
        OPTIONAL {{ ?player foaf:name ?playerName . }}

        OPTIONAL {{
            {{
                SELECT DISTINCT ?player ?team ?teamName ?teamCode ?teamYear
                                ?franchise ?franchiseName ?franchiseCode
                                ?leagueNode ?leagueCode
                WHERE {{
                    ?player bb:playerID "{player_id}" .
                    {{
                        ?player bb:hasBatting ?teamLine .
                    }}
                    UNION
                    {{
                        ?player bb:hasPitching ?teamLine .
```

```
            }}

            ?teamLine bb:teamOf ?team .
            OPTIONAL {{ ?team bb:teamName ?teamName . }}
            OPTIONAL {{ ?team bb:teamID ?teamCode . }}
            OPTIONAL {{ ?team bb:yearID ?teamYear . }}
            OPTIONAL {{
                ?team bb:franchiseOf ?franchise .
                OPTIONAL {{ ?franchise bb:franchiseName ?franchiseName . }}
                OPTIONAL {{ ?franchise bb:franchID ?franchiseCode . }}
            }}
            OPTIONAL {{
                ?team bb:lgID ?leagueCode .
                BIND(IRI(CONCAT("http://baseball.ws.pt/graph/league/", STR(?leagueCode))) AS
                ↪  ?leagueNode)
            }}
        }}
    }}
}}

OPTIONAL {{
    {{
        SELECT DISTINCT ?player ?award ?awardName ?awardYear
        WHERE {{
            ?player bb:playerID "{player_id}" ;
                    bb:wonAward ?award .
            OPTIONAL {{ ?award bb:awardName ?awardName . }}
            OPTIONAL {{ ?award bb:yearID ?awardYear . }}
        }}
    }}
}}

OPTIONAL {{
    {{
        SELECT ?player ?teammate ?teammateCode ?teammateName ?sharedSeasons
        WHERE {{
            {{
                SELECT ?player ?teammate ?teammateCode ?teammateName
                       (COUNT(DISTINCT ?sharedTeam) AS ?sharedSeasons)
                WHERE {{
                    ?player bb:playerID "{player_id}" .
                    {{
                        ?player bb:hasBatting ?playerLine .
                    }}
                    UNION
                    {{
                        ?player bb:hasPitching ?playerLine .
                    }}

                    ?playerLine bb:teamOf ?sharedTeam .

                    ?teammate a bb:Player ;
                              bb:playerID ?teammateCode .
                    FILTER(?teammate != ?player)
                    OPTIONAL {{ ?teammate foaf:name ?teammateName . }}

                    {{
                        ?teammate bb:hasBatting ?teammateLine .
                    }}
                    UNION
                    {{
                        ?teammate bb:hasPitching ?teammateLine .
                    }}

                    ?teammateLine bb:teamOf ?sharedTeam .
```

```
                    }}
                    GROUP BY ?player ?teammate ?teammateCode ?teammateName
                    ORDER BY DESC(?sharedSeasons) ?teammateName
                }}
            }}
        }}
    }}

    OPTIONAL {{
        {{
            SELECT DISTINCT ?team ?managerPlayer ?managerCode ?managerName
            WHERE {{
                ?player bb:playerID "{player_id}" .
                {{
                    ?player bb:hasBatting ?managerLine .
                }}
                UNION
                {{
                    ?player bb:hasPitching ?managerLine .
                }}

                ?managerLine bb:teamOf ?team .

                ?managerPlayer a bb:Player ;
                               bb:playerID ?managerCode ;
                               bb:isManager ?managerRole .
                OPTIONAL {{ ?managerPlayer foaf:name ?managerName . }}

                ?managerRole bb:managedTeam ?team .
            }}
        }}
    }}
}}
"""


def _coerce_year(value):
    try:
        return int(str(value).strip())
    except (TypeError, ValueError, AttributeError):
        return None


def _team_graph_construct_query(team_code, team_year):
    return f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    CONSTRUCT {{
        ?team a bb:GraphFocusTeam ;
              bb:teamName ?teamName ;
              bb:teamID ?teamCode ;
              bb:yearID ?teamYear ;
              bb:teamIDBR ?teamCodeBr .

        ?franchise a bb:GraphFranchise ;
                   bb:franchiseName ?franchiseName ;
                   bb:franchID ?franchiseCode .

        ?leagueNode a bb:GraphLeague ;
                    bb:lgID ?leagueCode .

        ?player a bb:GraphRosterPlayer ;
                bb:playerID ?playerCode ;
                bb:bbrefID ?playerBbrefID ;
```

```
            foaf:name ?playerName .

    ?managerPlayer a bb:GraphManager ;
                   bb:playerID ?managerCode ;
                   bb:bbrefID ?managerBbrefID ;
                   foaf:name ?managerName .

    ?award a bb:GraphAward ;
           bb:awardName ?awardName ;
           bb:yearID ?awardYear .

    ?team bb:franchiseLink ?franchise .
    ?team bb:leagueLink ?leagueNode .
    ?team bb:rosterLink ?player .
    ?team bb:managedByPerson ?managerPlayer .
    ?player bb:wonGraphAward ?award .
}}
WHERE {{
    ?team a bb:Team ;
          bb:teamID "{team_code}" ;
          bb:yearID {team_year} .
    OPTIONAL {{ ?team bb:teamName ?teamName . }}
    OPTIONAL {{ ?team bb:teamID ?teamCode . }}
    OPTIONAL {{ ?team bb:teamIDBR ?teamCodeBr . }}
    OPTIONAL {{ ?team bb:yearID ?teamYear . }}

    OPTIONAL {{
        ?team bb:franchiseOf ?franchise .
        OPTIONAL {{ ?franchise bb:franchiseName ?franchiseName . }}
        OPTIONAL {{ ?franchise bb:franchID ?franchiseCode . }}
    }}

    OPTIONAL {{
        ?team bb:lgID ?leagueCode .
        BIND(IRI(CONCAT("http://baseball.ws.pt/graph/league/", STR(?leagueCode))) AS ?leagueNode)
    }}

    OPTIONAL {{
        {{
            SELECT DISTINCT ?team ?player ?playerCode ?playerBbrefID ?playerName
            WHERE {{
                ?team a bb:Team ;
                      bb:teamID "{team_code}" ;
                      bb:yearID {team_year} .

                ?player a bb:Player ;
                        bb:playerID ?playerCode .
                OPTIONAL {{ ?player bb:bbrefID ?playerBbrefID . }}
                OPTIONAL {{ ?player foaf:name ?playerName . }}

                {{
                    ?player bb:hasBatting ?playerLine .
                }}
                UNION
                {{
                    ?player bb:hasPitching ?playerLine .
                }}

                ?playerLine bb:teamOf ?team .
            }}
        }}
    }}

    OPTIONAL {{
        {{
```

```
                SELECT DISTINCT ?team ?managerPlayer ?managerCode ?managerBbrefID ?managerName
                WHERE {{
                    ?team a bb:Team ;
                          bb:teamID "{team_code}" ;
                          bb:yearID {team_year} .

                    ?managerPlayer a bb:Player ;
                                   bb:playerID ?managerCode ;
                                   bb:isManager ?managerRole .
                    OPTIONAL {{ ?managerPlayer bb:bbrefID ?managerBbrefID . }}
                    OPTIONAL {{ ?managerPlayer foaf:name ?managerName . }}

                    ?managerRole bb:managedTeam ?team .
                }}
            }}
        }}

        OPTIONAL {{
            {{
                SELECT DISTINCT ?player ?award ?awardName ?awardYear
                WHERE {{
                    ?team a bb:Team ;
                          bb:teamID "{team_code}" ;
                          bb:yearID {team_year} .

                    ?player a bb:Player ;
                            bb:wonAward ?award .

                    {{
                        ?player bb:hasBatting ?awardLine .
                    }}
                    UNION
                    {{
                        ?player bb:hasPitching ?awardLine .
                    }}

                    ?awardLine bb:teamOf ?team .

                    OPTIONAL {{ ?award bb:awardName ?awardName . }}
                    OPTIONAL {{ ?award bb:yearID ?awardYear . }}
                    FILTER(!BOUND(?awardYear) || ?awardYear = {team_year})
                }}
            }}
        }}
    }}
    """


def _graph_to_cytoscape(graph):
    node_map = {}
    for subject in set(graph.subjects()):
        node_type = _choose_node_type(graph, subject)
        if not node_type:
            continue

        node_id = str(subject)
        node_data = {
            "id": node_id,
            "label": _build_node_label(graph, subject, node_type),
            "type": node_type,
        }

        if node_type in {"focus-team", "team"}:
            team_code = _literal_value(graph, subject, BB.teamID)
            if team_code:
```

```
                node_data["teamID"] = team_code
            team_code_br = _literal_value(graph, subject, BB.teamIDBR)
            if team_code_br:
                node_data["teamIDBR"] = team_code_br
        elif node_type in {"player", "manager", "teammate"}:
            player_code = _literal_value(graph, subject, BB.playerID)
            if player_code:
                node_data["playerID"] = player_code
            bbref_id = _literal_value(graph, subject, BB.bbrefID)
            if bbref_id:
                node_data["bbrefID"] = bbref_id
        elif node_type == "league":
            league_code = _literal_value(graph, subject, BB.lgID)
            if league_code:
                node_data["leagueCode"] = league_code
        if node_type == "teammate":
            shared_seasons = _literal_value(graph, subject, BB.sharedSeasons)
            if shared_seasons:
                node_data["sharedSeasons"] = shared_seasons

        node_map[node_id] = {"data": node_data}

    edges = []
    seen_edges = set()
    for predicate, label in EDGE_LABEL_MAP.items():
        for source, target in graph.subject_objects(predicate):
            source_id = str(source)
            target_id = str(target)
            if source_id not in node_map or target_id not in node_map:
                continue

            edge_key = (str(predicate), source_id, target_id)
            if edge_key in seen_edges:
                continue

            edges.append(
                {
                    "data": {
                        "id": f"edge:{len(edges)}",
                        "source": source_id,
                        "target": target_id,
                        "label": label,
                    }
                }
            )
            seen_edges.add(edge_key)

    return {
        "nodes": sorted(
            node_map.values(),
            key=lambda node: (NODE_TYPE_PRIORITY.get(node["data"]["type"], 999), node["data"]["label"]),
        ),
        "edges": edges,
    }


@lru_cache(maxsize=128)
def get_player_graph_data(player_id):
    player_id = escape_sparql_string(str(player_id).strip())
    if not player_id:
        return {"nodes": [], "edges": []}

    graph = run_construct(_player_graph_construct_query(player_id))
    if not graph:
        return {"nodes": [], "edges": []}
```

```
    return _graph_to_cytoscape(graph)


@lru_cache(maxsize=128)
def get_team_graph_data(team_code, year):
    team_code = escape_sparql_string(str(team_code).strip().upper())
    team_year = _coerce_year(year)
    if not team_code or team_year is None:
        return {"nodes": [], "edges": []}

    graph = run_construct(_team_graph_construct_query(team_code, team_year))
    if not graph:
        return {"nodes": [], "edges": []}

    return _graph_to_cytoscape(graph)
```

3.4.5 misc.py

```
from functools import lru_cache

from .base import _row_int, _row_value, run_query

@lru_cache(maxsize=1)
def get_top_salaries():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    SELECT ?name ?salary ?year WHERE {
        ?p a bb:Player ;
           foaf:name ?name ;
           bb:hasSalary ?s .
        ?s bb:salary ?salary ;
           bb:yearID ?year .
    }
    ORDER BY DESC(?salary)
    LIMIT 10
    """
    return run_query(query)


@lru_cache(maxsize=1)
def get_awards_list():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?awardName ?year ?lg
    WHERE {
        ?p bb:wonAward ?awObj .
        ?p bb:playerID ?playerID .
        ?awObj bb:awardName ?awardName ;
              bb:yearID ?year .

        OPTIONAL { ?p foaf:name ?name . }
        OPTIONAL { ?p bb:nameFirst ?nameFirst . }
        OPTIONAL { ?p bb:nameLast ?nameLast . }
        OPTIONAL { ?p bb:nameGiven ?nameGiven . }
        OPTIONAL { ?awObj bb:lgID ?lg . }
    }
    ORDER BY DESC(?year)
    LIMIT 25
    """
    awards = []
```

```
    for row in run_query(query):
        first_name = _row_value(row, "nameFirst", "")
        last_name = _row_value(row, "nameLast", "")
        full_name = " ".join(part for part in [first_name, last_name] if part).strip()

        if not full_name:
            full_name = _row_value(row, "name", "")
        if not full_name:
            full_name = _row_value(row, "nameGiven", "")
        if not full_name:
            full_name = _row_value(row, "playerID", "Unknown player")

        awards.append(
            {
                "player_id": _row_value(row, "playerID", ""),
                "name": full_name,
                "award_name": _row_value(row, "awardName", "Award"),
                "year": _row_int(row, "year", 0),
                "league": _row_value(row, "lg", ""),
            }
        )

    return awards


def get_header_teams_graph():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT DISTINCT ?teamCode ?teamName ?franchise
    WHERE {
        ?team a bb:Team ;
              bb:yearID ?year ;
              bb:teamName ?teamName .
        FILTER(?year = 2015)
        OPTIONAL { ?team bb:teamIDBR ?teamCodeBR . }
        OPTIONAL { ?team bb:teamID ?teamCodeID . }
        OPTIONAL { ?team bb:franchID ?franchise . }
        BIND(
            COALESCE(
                ?teamCodeBR,
                ?teamCodeID,
                STRBEFORE(STRAFTER(STR(?team), "/team/"), "/")
            ) AS ?teamCode
        )
    }
    ORDER BY ?teamCode
    """

    return [
        {
            "abbr": row.get("teamCode", {}).get("value", ""),
            "name": row.get("teamName", {}).get("value", row.get("teamCode", {}).get("value", "")),
            "franchise": row.get("franchise", {}).get("value", row.get("teamCode", {}).get("value", "")),
        }
        for row in run_query(query)
        if row.get("teamCode", {}).get("value")
    ]

@lru_cache(maxsize=1)
def get_hall_of_fame_members():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
```

```
SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?yearID ?votedBy ?ballots ?votes ?needed
↪  ?category
WHERE {
    ?hof a bb:HallOfFameVote ;
         bb:inducted true ;
         bb:yearID ?yearID ;
         bb:category ?category .

    # Para esta lista, queremos focar em quem tem dados de votos
    ?hof bb:votes ?votes .
    ?hof bb:ballots ?ballots .

    OPTIONAL { ?hof bb:votedBy ?votedBy . }
    OPTIONAL { ?hof bb:needed ?needed . }

    ?player bb:hallOfFameVote ?hof ;
            bb:playerID ?playerID .

    OPTIONAL { ?player foaf:name ?name . }
    OPTIONAL { ?player bb:nameFirst ?nameFirst . }
    OPTIONAL { ?player bb:nameLast ?nameLast . }
    OPTIONAL { ?player bb:nameGiven ?nameGiven . }
}
"""
results = []
seen = set()

for row in run_query(query):
    pid = _row_value(row, "playerID")
    year = _row_value(row, "yearID")
    key = f"{pid}_{year}"

    if key in seen:
        continue
    seen.add(key)

    # Construção do nome
    first_name = _row_value(row, "nameFirst", "")
    last_name = _row_value(row, "nameLast", "")
    full_name = " ".join(part for part in [first_name, last_name] if part).strip()

    if not full_name:
        full_name = _row_value(row, "name", "")
    if not full_name:
        full_name = _row_value(row, "nameGiven", pid)

    # Dados numéricos
    votes = _row_int(row, "votes", 0)
    ballots = _row_int(row, "ballots", 0)
    needed = _row_int(row, "needed", 0)

    # Cálculo da percentagem (apenas se ballots > 0)
    if ballots > 0:
        percent_val = (votes / ballots * 100)

        results.append({
            "player_id": pid,
            "name": full_name,
            "year": _row_int(row, "yearID", 0),
            "voted_by": _row_value(row, "votedBy", "-"),
            "category": _row_value(row, "category", "-"),
            "votes_str": f"{votes} / {ballots}",
            "needed_str": str(needed) if needed > 0 else "-",
            "percent": f"{percent_val:.1f}%",
            "sort_percent": percent_val
```

```
            })

    # Ordenação: Primeiro os que tiveram maior % de votos, depois por ano mais recente
    results.sort(key=lambda x: (x["sort_percent"], x["year"]), reverse=True)

    return results

@lru_cache(maxsize=1)
def get_managers_list():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name (SUM(?w) AS ?totalWins) (SUM(?l) AS
    ↪  ?totalLosses) (MIN(?yearID) AS ?firstYear) (MAX(?yearID) AS ?lastYear)
    WHERE {
        ?mgr a bb:Manager ;
             bb:yearID ?yearID ;
             bb:wins ?w ;
             bb:losses ?l .

        ?player bb:isManager ?mgr ;
                bb:playerID ?playerID .

        OPTIONAL { ?player foaf:name ?name . }
        OPTIONAL { ?player bb:nameFirst ?nameFirst . }
        OPTIONAL { ?player bb:nameLast ?nameLast . }
        OPTIONAL { ?player bb:nameGiven ?nameGiven . }
    }
    GROUP BY ?playerID ?nameFirst ?nameLast ?nameGiven ?name
    HAVING (SUM(?w) > 0)
    ORDER BY DESC(?totalWins)
    LIMIT 200
    """
    results = []
    for row in run_query(query):
        pid = _row_value(row, "playerID")

        first_name = _row_value(row, "nameFirst", "")
        last_name = _row_value(row, "nameLast", "")
        full_name = " ".join(part for part in [first_name, last_name] if part).strip()
        if not full_name:
            full_name = _row_value(row, "name", "")
        if not full_name:
            full_name = _row_value(row, "nameGiven", pid)

        wins = _row_int(row, "totalWins", 0)
        losses = _row_int(row, "totalLosses", 0)
        total_games = wins + losses
        win_pct = (wins / total_games) if total_games > 0 else 0

        results.append({
            "player_id": pid,
            "name": full_name,
            "wins": wins,
            "losses": losses,
            "win_pct": f"{win_pct:.3f}",
            "first_year": _row_int(row, "firstYear", 0),
            "last_year": _row_int(row, "lastYear", 0)
        })
    return results

@lru_cache(maxsize=1)
def get_global_player_leaders():
    # Helper for top sum queries
```

```
    def _get_top_sum(stat_type_class, stat_property, has_relation):
        query = f"""
        PREFIX bb: <http://baseball.ws.pt/>
        PREFIX foaf: <http://xmlns.com/foaf/0.1/>

        SELECT ?playerID ?nameFirst ?nameLast ?nameGiven ?name (SUM(?statValue) AS ?totalStat)
        WHERE {{
            ?stat a bb:{stat_type_class} ;
                  bb:{stat_property} ?statValue .
            ?player bb:{has_relation} ?stat ;
                    bb:playerID ?playerID .

            OPTIONAL {{ ?player foaf:name ?name . }}
            OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
            OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
            OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
        }}
        GROUP BY ?playerID ?nameFirst ?nameLast ?nameGiven ?name
        ORDER BY DESC(?totalStat)
        LIMIT 5
        """
        results = []
        for row in run_query(query):
            first_name = _row_value(row, "nameFirst", "")
            last_name = _row_value(row, "nameLast", "")
            full_name = " ".join(part for part in [first_name, last_name] if part).strip()
            if not full_name:
                full_name = _row_value(row, "name", "")
            if not full_name:
                full_name = _row_value(row, "nameGiven", _row_value(row, "playerID", ""))

            results.append({
                "name": full_name,
                "player_id": _row_value(row, "playerID", ""),
                "value": _row_int(row, "totalStat", 0)
            })
        return results

    return {
        "hr": _get_top_sum("BattingStat", "homeRuns", "hasBatting"),
        "rbi": _get_top_sum("BattingStat", "RBI", "hasBatting"),
        "strikeouts": _get_top_sum("PitchingStat", "SO", "hasPitching"),
    }

@lru_cache(maxsize=1)
def get_global_team_leaders():
    query = """
    PREFIX bb: <http://baseball.ws.pt/>

    SELECT ?franchID ?franchName (SUM(?w) AS ?totalWins)
    WHERE {
        ?team a bb:Team ;
              bb:franchID ?franchID ;
              bb:W ?w .
        ?franch a bb:Franchise ;
                bb:franchID ?franchID ;
                bb:franchiseName ?franchName .
    }
    GROUP BY ?franchID ?franchName
    ORDER BY DESC(?totalWins)
    LIMIT 10
    """
    results = []
    for row in run_query(query):
        results.append({
```

```
            "franch_id": _row_value(row, "franchID", ""),
            "name": _row_value(row, "franchName", ""),
            "wins": _row_int(row, "totalWins", 0)
        })
    return results
```

3.4.6 quiz.py

```
from functools import lru_cache

from .base import _row_int, _row_value, escape_sparql_string, run_ask, run_query
from .leagues import LEAGUE_LABELS


QUIZ_YEAR_MIN = 2000


def _player_name_from_row(row):
    first_name = _row_value(row, "nameFirst", "")
    last_name = _row_value(row, "nameLast", "")
    full_name = " ".join(part for part in [first_name, last_name] if part).strip()
    if not full_name:
        full_name = _row_value(row, "name", "")
    if not full_name:
        full_name = _row_value(row, "nameGiven", _row_value(row, "playerID", ""))
    return full_name


@lru_cache(maxsize=8)
def get_quiz_leaderboard_bank(stat_key):
    stat_map = {
        "home_runs": ("BattingStat", "homeRuns", "hasBatting"),
        "rbi": ("BattingStat", "RBI", "hasBatting"),
        "strikeouts": ("PitchingStat", "SO", "hasPitching"),
    }
    stat_config = stat_map.get(str(stat_key).strip())
    if not stat_config:
        return []

    stat_type_class, stat_property, has_relation = stat_config
    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?lg ?year ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?statValue
    WHERE {{
        ?stat a bb:{stat_type_class} ;
              bb:lgID ?lg ;
              bb:yearID ?year ;
              bb:{stat_property} ?statValue .
        ?player bb:{has_relation} ?stat ;
                bb:playerID ?playerID .
        FILTER(?year >= {QUIZ_YEAR_MIN})
        FILTER(?lg IN ("AL", "NL"))

        OPTIONAL {{ ?player foaf:name ?name . }}
        OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
        OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
        OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
    }}
    ORDER BY ?lg DESC(?year) DESC(?statValue) ?playerID
    """

    grouped = {}
```

```
    for row in run_query(query):
        league_code = _row_value(row, "lg", "").strip().upper()
        year = _row_int(row, "year", None)
        player_id = _row_value(row, "playerID", "").strip()
        if not league_code or not year or not player_id:
            continue

        key = (league_code, year)
        bucket = grouped.setdefault(
            key,
            {
                "league_code": league_code,
                "year": year,
                "league_label": LEAGUE_LABELS.get(league_code, league_code),
                "leaders": [],
                "_seen_players": set(),
            },
        )
        if player_id in bucket["_seen_players"]:
            continue

        bucket["_seen_players"].add(player_id)
        bucket["leaders"].append(
            {
                "player_id": player_id,
                "name": _player_name_from_row(row),
                "value": _row_int(row, "statValue", 0),
            }
        )

    records = []
    for item in grouped.values():
        if len(item["leaders"]) < 4:
            continue
        records.append(
            {
                "league_code": item["league_code"],
                "league_label": item["league_label"],
                "year": item["year"],
                "leaders": item["leaders"][:4],
            }
        )

    return sorted(records, key=lambda item: (item["league_code"], -item["year"]))


@lru_cache(maxsize=1)
def get_quiz_salary_bank():
    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?year ?playerID ?nameFirst ?nameLast ?nameGiven ?name ?salary
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                bb:hasSalary ?salaryObj .
        ?salaryObj bb:salary ?salary ;
                   bb:yearID ?year .
        FILTER(?year >= {QUIZ_YEAR_MIN})

        OPTIONAL {{ ?player foaf:name ?name . }}
        OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
        OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
        OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
```

```
    }}
    ORDER BY DESC(?year) DESC(?salary) ?playerID
    """

    grouped = {}
    for row in run_query(query):
        year = _row_int(row, "year", None)
        player_id = _row_value(row, "playerID", "").strip()
        if not year or not player_id:
            continue

        bucket = grouped.setdefault(
            year,
            {
                "year": year,
                "leaders": [],
                "_seen_players": set(),
            },
        )
        if player_id in bucket["_seen_players"]:
            continue

        bucket["_seen_players"].add(player_id)
        bucket["leaders"].append(
            {
                "player_id": player_id,
                "name": _player_name_from_row(row),
                "value": _row_int(row, "salary", 0),
            }
        )

    records = []
    for item in grouped.values():
        if len(item["leaders"]) < 4:
            continue
        records.append(
            {
                "year": item["year"],
                "leaders": item["leaders"][:4],
            }
        )

    return sorted(records, key=lambda item: -item["year"])


@lru_cache(maxsize=1)
def get_quiz_award_bank():
    supported_awards = [
        "Most Valuable Player",
        "Cy Young Award",
        "Rookie of the Year",
        "Hank Aaron Award",
        "Comeback Player of the Year",
    ]
    award_filter = ", ".join(f'"{escape_sparql_string(name)}"' for name in supported_awards)

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>

    SELECT ?awardName ?lg ?year ?playerID ?nameFirst ?nameLast ?nameGiven ?name
    WHERE {{
        ?player a bb:Player ;
                bb:playerID ?playerID ;
                bb:wonAward ?award .
```

```
    ?award bb:awardName ?awardName ;
           bb:yearID ?year .
    FILTER(?year >= {QUIZ_YEAR_MIN})
    FILTER(?awardName IN ({award_filter}))
    OPTIONAL {{ ?award bb:lgID ?lg . }}
    OPTIONAL {{ ?player foaf:name ?name . }}
    OPTIONAL {{ ?player bb:nameFirst ?nameFirst . }}
    OPTIONAL {{ ?player bb:nameLast ?nameLast . }}
    OPTIONAL {{ ?player bb:nameGiven ?nameGiven . }}
}}
ORDER BY ?awardName ?lg ?year ?playerID
"""

grouped = {}
for row in run_query(query):
    award_name = _row_value(row, "awardName", "").strip()
    league_code = _row_value(row, "lg", "").strip().upper() or "ML"
    year = _row_int(row, "year", None)
    player_id = _row_value(row, "playerID", "").strip()
    if not award_name or not year or not player_id:
        continue

    group_key = (award_name, league_code)
    bucket = grouped.setdefault(
        group_key,
        {
            "award_name": award_name,
            "league_code": league_code,
            "winners_by_year": {},
        },
    )
    winners = bucket["winners_by_year"].setdefault(year, [])
    winners.append(
        {
            "year": year,
            "player_id": player_id,
            "name": _player_name_from_row(row),
        }
    )

records = []
for item in grouped.values():
    winners = []
    for year, year_winners in item["winners_by_year"].items():
        unique_by_player = []
        seen_players = set()
        for winner in year_winners:
            if winner["player_id"] in seen_players:
                continue
            seen_players.add(winner["player_id"])
            unique_by_player.append(winner)
        if len(unique_by_player) == 1:
            winners.append(unique_by_player[0])

    if len(winners) < 4:
        continue

    records.append(
        {
            "award_name": item["award_name"],
            "league_code": item["league_code"],
            "winners": sorted(winners, key=lambda winner: winner["year"]),
        }
    )
```

```
    return sorted(records, key=lambda item: (item["award_name"], item["league_code"]))


def ask_quiz_leaderboard_answer(stat_key, league_code, year, player_id):
    stat_map = {
        "home_runs": ("BattingStat", "homeRuns", "hasBatting"),
        "rbi": ("BattingStat", "RBI", "hasBatting"),
        "strikeouts": ("PitchingStat", "SO", "hasPitching"),
    }
    stat_config = stat_map.get(str(stat_key).strip())
    if not stat_config:
        return False

    stat_type_class, stat_property, has_relation = stat_config
    safe_league = escape_sparql_string(str(league_code).strip().upper())
    safe_player = escape_sparql_string(str(player_id).strip())
    try:
        year_value = int(year)
    except (TypeError, ValueError):
        return False

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    ASK {{
        ?player bb:playerID "{safe_player}" ;
                bb:{has_relation} ?stat .
        ?stat a bb:{stat_type_class} ;
              bb:lgID "{safe_league}" ;
              bb:yearID {year_value} ;
              bb:{stat_property} ?statValue .

        FILTER NOT EXISTS {{
            ?otherStat a bb:{stat_type_class} ;
                       bb:lgID "{safe_league}" ;
                       bb:yearID {year_value} ;
                       bb:{stat_property} ?otherValue .
            FILTER(?otherValue > ?statValue)
        }}
    }}
    """
    return run_ask(query)


def ask_quiz_salary_answer(year, player_id):
    safe_player = escape_sparql_string(str(player_id).strip())
    try:
        year_value = int(year)
    except (TypeError, ValueError):
        return False

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    ASK {{
        ?player a bb:Player ;
                bb:playerID "{safe_player}" ;
                bb:hasSalary ?salaryObj .
        ?salaryObj bb:salary ?salaryValue ;
                   bb:yearID {year_value} .

        FILTER NOT EXISTS {{
            ?otherPlayer a bb:Player ;
                         bb:hasSalary ?otherSalaryObj .
            ?otherSalaryObj bb:salary ?otherValue ;
```

```
                        bb:yearID {year_value} .
            FILTER(?otherValue > ?salaryValue)
        }}
    }}
    """
    return run_ask(query)


def ask_quiz_award_answer(award_name, league_code, year, player_id):
    safe_award = escape_sparql_string(str(award_name).strip())
    safe_league = escape_sparql_string(str(league_code or "").strip().upper())
    safe_player = escape_sparql_string(str(player_id).strip())
    try:
        year_value = int(year)
    except (TypeError, ValueError):
        return False

    league_filter = ""
    if safe_league and safe_league != "ML":
        league_filter = f'?award bb:lgID "{safe_league}" .'

    query = f"""
    PREFIX bb: <http://baseball.ws.pt/>

    ASK {{
        ?player a bb:Player ;
                bb:playerID "{safe_player}" ;
                bb:wonAward ?award .
        ?award bb:awardName "{safe_award}" ;
               bb:yearID {year_value} .
        {league_filter}
    }}
    """
    return run_ask(query)
```

3.4.7 edit_service.py

```
from collections import OrderedDict
from datetime import date
from urllib.error import HTTPError, URLError
from urllib.parse import urlsplit, urlunsplit, urlencode
from urllib.request import Request, urlopen

from django.utils import timezone

from .models import DataSuggestion, DataSuggestionChange
from .sparql import DIVISION_LABELS, ENDPOINT, LEAGUE_LABELS, escape_sparql_string


BB_NS = "http://baseball.ws.pt/"
XSD_NS = "http://www.w3.org/2001/XMLSchema#"
GRAPHDB_UPDATE_ENDPOINT = urlunsplit((*urlsplit(ENDPOINT)[:2],
↪  f"{urlsplit(ENDPOINT).path.rstrip('/')}/statements", "", ""))


PLAYER_FIELD_SPECS = OrderedDict(
    [
        ("display_name", {"label": "Player name", "type": "text", "predicate": "foaf:name", "scope":
        ↪  "player"}),
        ("birth_country", {"label": "Birth country", "type": "text", "predicate": "bb:birthCountry",
        ↪  "scope": "player"}),
        ("birth_state", {"label": "Birth state", "type": "text", "predicate": "bb:birthState", "scope":
        ↪  "player"}),
```

```
        ("birth_city", {"label": "Birth city", "type": "text", "predicate": "bb:birthCity", "scope":
        ↪  "player"}),
        (
            "bats",
            {
                "label": "Bats",
                "type": "choice",
                "predicate": "bb:bats",
                "scope": "player",
                "choices": [("R", "Right"), ("L", "Left"), ("B", "Switch"), ("", "Unknown")],
            },
        ),
        (
            "throws",
            {
                "label": "Throws",
                "type": "choice",
                "predicate": "bb:throws",
                "scope": "player",
                "choices": [("R", "Right"), ("L", "Left"), ("", "Unknown")],
            },
        ),
        ("debut", {"label": "Debut", "type": "date", "predicate": "bb:debut", "scope": "player"}),
        ("final_game", {"label": "Final game", "type": "date", "predicate": "bb:finalGame", "scope":
        ↪  "player"}),
    ]
)

TEAM_FIELD_SPECS = OrderedDict(
    [
        ("team_name", {"label": "Team name", "type": "text", "predicate": "bb:teamName", "scope": "team"}),
        ("franchise_name", {"label": "Franchise name", "type": "text", "predicate": "bb:franchiseName",
        ↪  "scope": "franchise"}),
        ("park", {"label": "Ballpark", "type": "text", "predicate": "bb:park", "scope": "team"}),
        (
            "league_code",
            {
                "label": "League",
                "type": "choice",
                "predicate": "bb:lgID",
                "scope": "team",
                "choices": [(code, name) for code, name in sorted(LEAGUE_LABELS.items())] + [("",
                ↪  "Unknown")],
            },
        ),
        (
            "division_code",
            {
                "label": "Division",
                "type": "choice",
                "predicate": "bb:divID",
                "scope": "team",
                "choices": [(code, name) for code, name in sorted(DIVISION_LABELS.items())] + [("",
                ↪  "Unknown")],
            },
        ),
        ("attendance", {"label": "Attendance", "type": "integer", "predicate": "bb:attendance", "scope":
        ↪  "team"}),
    ]
)

FIELD_SPECS = {
    DataSuggestion.ENTITY_PLAYER: PLAYER_FIELD_SPECS,
    DataSuggestion.ENTITY_TEAM: TEAM_FIELD_SPECS,
}
```

```
class SuggestionValidationError(ValueError):
    pass


def get_field_specs(entity_type):
    return FIELD_SPECS.get(entity_type, OrderedDict())


def build_player_edit_state(profile, is_admin):
    current_values = {
        "display_name": str(profile.get("name") or ""),
        "birth_country": str(profile.get("birth_country") or ""),
        "birth_state": str(profile.get("birth_state") or ""),
        "birth_city": str(profile.get("birth_city") or ""),
        "bats": str(profile.get("bats") or ""),
        "throws": str(profile.get("throws") or ""),
        "debut": str(profile.get("debut") or ""),
        "final_game": str(profile.get("final_game") or ""),
    }
    return _build_edit_state(
        entity_type=DataSuggestion.ENTITY_PLAYER,
        entity_id=str(profile.get("player_id") or ""),
        entity_year=None,
        current_values=current_values,
        is_admin=is_admin,
    )


def build_team_edit_state(team, is_admin):
    current_values = {
        "team_name": str(team.get("team_name") or ""),
        "franchise_name": str(team.get("franchise_name") or ""),
        "park": str(team.get("park") or ""),
        "league_code": str(team.get("league_code") or ""),
        "division_code": str(team.get("division_code") or ""),
        "attendance": "" if team.get("attendance") in (None, "") else str(int(team.get("attendance"))),
    }
    return _build_edit_state(
        entity_type=DataSuggestion.ENTITY_TEAM,
        entity_id=str(team.get("franchise_id") or ""),
        entity_year=team.get("year"),
        current_values=current_values,
        is_admin=is_admin,
    )


def _build_edit_state(*, entity_type, entity_id, entity_year, current_values, is_admin):
    fields = []
    for field_key, spec in get_field_specs(entity_type).items():
        fields.append(
            {
                "key": field_key,
                "label": spec["label"],
                "type": spec["type"],
                "value": current_values.get(field_key, ""),
                "display": display_field_value(entity_type, field_key, current_values.get(field_key, "")),
                "choices": [{"value": value, "label": label} for value, label in spec.get("choices", [])],
            }
        )
    return {
        "entity_type": entity_type,
        "entity_id": entity_id,
        "entity_year": entity_year,
```

```
        "is_admin": bool(is_admin),
        "fields": fields,
        "current_values": current_values,
    }


def display_field_value(entity_type, field_key, value):
    spec = get_field_specs(entity_type).get(field_key)
    if not spec:
        return str(value or "")

    if value in (None, ""):
        return "N/A"

    if spec["type"] == "choice":
        return dict(spec.get("choices", [])).get(value, value)
    if spec["type"] == "integer":
        try:
            return f"{int(value):,}"
        except (TypeError, ValueError):
            return str(value)
    return str(value)


def normalize_submitted_changes(entity_type, raw_changes, current_values):
    specs = get_field_specs(entity_type)
    prepared = []

    if not isinstance(raw_changes, dict):
        raise SuggestionValidationError("Changes payload is invalid.")

    for field_key, raw_value in raw_changes.items():
        if field_key not in specs:
            raise SuggestionValidationError(f"Field '{field_key}' cannot be edited.")

        normalized_value = _normalize_field_value(specs[field_key], raw_value)
        current_value = str(current_values.get(field_key) or "")
        if normalized_value == current_value:
            continue

        prepared.append(
            {
                "field_key": field_key,
                "label": specs[field_key]["label"],
                "old_value": current_value,
                "new_value": normalized_value,
                "old_display": display_field_value(entity_type, field_key, current_value),
                "new_display": display_field_value(entity_type, field_key, normalized_value),
            }
        )

    if not prepared:
        raise SuggestionValidationError("No valid field changes were submitted.")

    return prepared


def _normalize_field_value(spec, raw_value):
    text = "" if raw_value is None else str(raw_value).strip()

    if spec["type"] == "text":
        return text
    if spec["type"] == "choice":
        allowed = {value for value, _label in spec.get("choices", [])}
        if text not in allowed:
```

```
            raise SuggestionValidationError(f"Invalid value for {spec['label']}.")
        return text
    if spec["type"] == "date":
        if not text:
            return ""
        try:
            date.fromisoformat(text)
        except ValueError as exc:
            raise SuggestionValidationError(f"Invalid date for {spec['label']}.") from exc
        return text
    if spec["type"] == "integer":
        if not text:
            return ""
        try:
            value = int(text)
        except ValueError as exc:
            raise SuggestionValidationError(f"Invalid number for {spec['label']}.") from exc
        if value < 0:
            raise SuggestionValidationError(f"{spec['label']} cannot be negative.")
        return str(value)
    return text


def create_suggestion(*, submitted_by, entity_type, entity_id, entity_year, reason, changes,
↪  status=DataSuggestion.STATUS_PENDING, reviewed_by=None, review_note=""):
    suggestion = DataSuggestion.objects.create(
        entity_type=entity_type,
        entity_id=entity_id,
        entity_year=entity_year,
        submitted_by=submitted_by,
        reason=reason,
        status=status,
        reviewed_by=reviewed_by,
        reviewed_at=timezone.now() if status != DataSuggestion.STATUS_PENDING else None,
        review_note=review_note,
    )
    DataSuggestionChange.objects.bulk_create(
        [
            DataSuggestionChange(
                suggestion=suggestion,
                field_key=change["field_key"],
                old_value=change["old_value"],
                new_value=change["new_value"],
            )
            for change in changes
        ]
    )
    return suggestion


def apply_suggestion_updates(entity_type, entity_id, entity_year, changes):
    if entity_type not in FIELD_SPECS:
        raise SuggestionValidationError("Unsupported entity type.")

    for change in changes:
        query = _build_update_query(
            entity_type=entity_type,
            entity_id=entity_id,
            entity_year=entity_year,
            field_key=change["field_key"],
            value=change["new_value"],
        )
        _run_graphdb_update(query)

    clear_sparql_caches()
```

```
def _build_update_query(*, entity_type, entity_id, entity_year, field_key, value):
    spec = get_field_specs(entity_type)[field_key]
    predicate = spec["predicate"]
    subject_selector = _subject_selector(entity_type, spec["scope"], entity_id, entity_year)
    current_var = "?currentValue"
    literal = _sparql_literal(spec, value)

    if literal is None:
        insert_block = ""
    else:
        insert_block = f"INSERT {{ ?subject {predicate} {literal} . }}\n"

    return f"""
PREFIX bb: <{BB_NS}>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
PREFIX xsd: <{XSD_NS}>

DELETE {{ ?subject {predicate} {current_var} . }}
{insert_block}WHERE {{
  {subject_selector}
  OPTIONAL {{ ?subject {predicate} {current_var} . }}
}}
""".strip()


def _subject_selector(entity_type, scope, entity_id, entity_year):
    safe_id = escape_sparql_string(entity_id)

    if entity_type == DataSuggestion.ENTITY_PLAYER and scope == "player":
        return f'?subject a bb:Player ; bb:playerID "{safe_id}" .'

    if entity_type == DataSuggestion.ENTITY_TEAM and scope == "franchise":
        return f'?subject a bb:Franchise ; bb:franchID "{safe_id}" .'

    if entity_type == DataSuggestion.ENTITY_TEAM and scope == "team":
        if not entity_year:
            raise SuggestionValidationError("Season year is required for team edits.")
        safe_year = escape_sparql_string(str(entity_year))
        return (
            f'?franchise a bb:Franchise ; bb:franchID "{safe_id}" . '
            f'?subject a bb:Team ; bb:franchiseOf ?franchise ; bb:yearID ?seasonYear . '
            f'FILTER(STR(?seasonYear) = "{safe_year}")'
        )

    raise SuggestionValidationError("Invalid update target.")


def _sparql_literal(spec, value):
    if value in (None, ""):
        return None

    safe_value = escape_sparql_string(value)
    if spec["type"] == "date":
        return f'"{safe_value}"^^xsd:date'
    if spec["type"] == "integer":
        return f'"{safe_value}"^^xsd:integer'
    return f'"{safe_value}"'


def _run_graphdb_update(query):
    payload = urlencode({"update": query}).encode("utf-8")
    request = Request(
        GRAPHDB_UPDATE_ENDPOINT,
```

```
        data=payload,
        headers={"Content-Type": "application/x-www-form-urlencoded; charset=UTF-8"},
        method="POST",
    )
    try:
        with urlopen(request, timeout=30):
            return
    except (HTTPError, URLError) as exc:
        raise RuntimeError("GraphDB update failed.") from exc


def clear_sparql_caches():
    from . import sparql
    from . import team_branding

    for module in (sparql, team_branding):
        for attribute_name in dir(module):
            attribute = getattr(module, attribute_name)
            if hasattr(attribute, "cache_clear"):
                try:
                    attribute.cache_clear()
                except Exception:
                    continue
```

3.5 Síntese

O sistema usa três tipos principais de operações SPARQL:

- `SELECT`, para construir catálogos, detalhes, rankings, históricos e grafos;
- `ASK`, para validar respostas no quiz diretamente contra o GraphDB;
- `DELETE/INSERT`, para publicar edições aprovadas por administradores.

Esta organização torna o relatório mais claro, aproxima a documentação da implementação real e evidencia como o projeto explora o GraphDB não apenas para leitura, mas também para validação e atualização de dados.

Capítulo 4

Funcionalidades da Aplicação (UI)

Descreva as funcionalidades de interface e a navegação do utilizador.

Sugestão

Insira as capturas de ecrã numeradas para reproduzir a lista de figuras (4.1 a 4.24).

Capítulo 5

Funcionalidades de Administrador (Autenticação e Operações CRUD)

Explique aqui o fluxo de autenticação e as operações CRUD disponíveis para administradores.

Capítulo 6

Conclusões

Resuma os resultados, desafios encontrados e melhorias futuras.

Capítulo 7

Configurações para executar a app

7.1 Visão geral

Para executar o projeto BaseBallX são necessários três elementos principais:

- um ambiente Python com as dependências da aplicação;
- um repositório `baseball` ativo no GraphDB;
- a aplicação Django a correr localmente.

O fluxo de execução pode ser resumido da seguinte forma:

1. criar e ativar o ambiente virtual Python;
2. instalar dependências;
3. gerar o ficheiro RDF `rdf/baseball.n3`;
4. iniciar o GraphDB e carregar o repositório `baseball`;
5. aplicar as migrations do Django;
6. iniciar o servidor web.

7.2 Pré-requisitos

Antes da execução, o sistema deve ter instalado:

- **Python 3**;
- **GraphDB Desktop**
- **curl**, usado pelos scripts auxiliares;
- acesso ao diretório do projeto com os dados na pasta `archive/`.

Garantir que a porta `7200` está livre para o GraphDB `http://localhost:7200`

7.3 Execução manual

7.3.1 1. Criar o ambiente virtual

Na raiz do projeto:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

7.3.2 2. Gerar o RDF

Se o ficheiro RDF ainda não existir deve ser executada a conversão:

```
python3 rdf/convert.py
```

Este comando gera o ficheiro:

`rdf/baseball.n3`

7.3.3 3. Iniciar o GraphDB e criar o repositório

Existem duas formas de preparar o GraphDB.

Opção A: usar o script auxiliar

```
./scripts/start_graphdb.sh
```

Este script:

- verifica se o GraphDB está acessível;
- tenta abrir o GraphDB Desktop se o serviço ainda não estiver ativo;
- cria o repositório `baseball` se este ainda não existir;
- importa automaticamente o ficheiro `rdf/baseball.n3`.

Por omissão, o script usa:

- `GRAPHDB_URL=http://localhost:7200`
- `REPOSITORY_ID=baseball`
- `RDF_FILE=$ROOT_DIR/rdf/baseball.n3`
- `GRAPHDB_DESKTOP_BIN=/opt/graphdb-desktop/bin/graphdb-desktop`

A aplicação usa SQLite para persistir contas, sugestões, sessões e resultados do quiz. Depois de ativar o ambiente virtual, devem ser aplicadas as migrations:

```
cd webapp
python manage.py migrate
```

7.3.4 5. Iniciar o servidor web

Ainda na pasta `webapp`, iniciar a aplicação Django:

```
python manage.py runserver
```

Depois disso, a aplicação fica disponível em:

`http://127.0.0.1:8000/`

7.4 Fluxo completo recomendado

O fluxo mais direto para executar o projeto desde o início é:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 rdf/convert.py
./scripts/start_graphdb.sh
cd webapp
python manage.py migrate
python manage.py runserver
```

7.5 Parar e reconstruir o repositório

Se for necessário apagar o repositório do GraphDB e voltar a importá-lo, existe um script dedicado:

```
./scripts/delete_graphdb_repo.sh
```

Depois da remoção, basta correr novamente:

```
./scripts/start_graphdb.sh
```

Isto volta a criar o repositório `baseball` e a importar o ficheiro RDF.

7.6 Execução com Docker

Nesta versão do projeto não foi incluída uma configuração Docker operacional, isto é, não existem `Dockerfile` nem `docker-compose.yml` prontos a usar no repositório. Assim, a forma suportada de execução é a execução manual descrita acima.

Se no futuro for pretendida uma distribuição por contentores, a configuração natural seria separar:

- um contentor para a aplicação Django;
- um contentor ou serviço externo para o GraphDB;
- um volume persistente para o repositório RDF;
- um volume persistente para a base de dados SQLite, ou a sua substituição por uma base de dados mais adequada a ambientes conteinerizados.

7.7 Notas finais de execução

Durante a execução, os problemas mais prováveis são:

- o GraphDB não estar ativo em `http://localhost:7200`;
- o repositório `baseball` não existir ou estar vazio;
- o ficheiro `rdf/baseball.n3` ainda não ter sido gerado;
- as migrations do Django não terem sido aplicadas;
- o ambiente virtual não estar ativado.

Se estes pontos estiverem corretamente configurados, a aplicação arranca sem dependências adicionais.